

K8s 클러스터 구성

inky4832@daum.net

1.1 실습 환경 구성

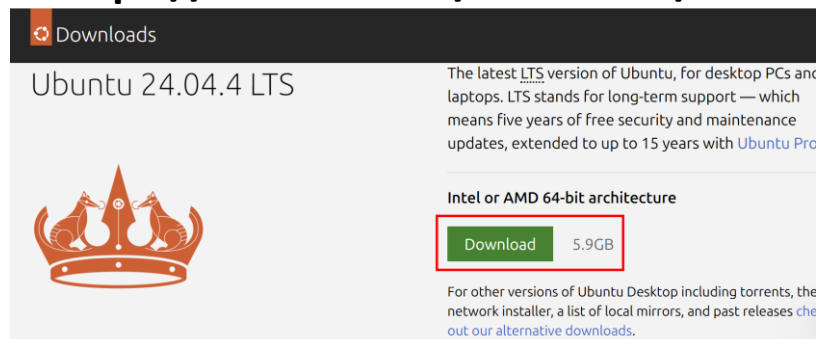
가상 머신

- PC에 설치되어 있는 운영체제(**호스트 OS**)에 가상의 머신(시스템)을 생성한 후 여기에 다른 운영체제(**게스트 OS**)를 설치할 수 있도록 해주는 응용 프로그램

가상머신	호스트OS	게스트OS
VMWare	윈도 계열, 대부분의 리눅스, 맥OS	윈도 계열OS, 대부분의 리눅스, 솔라리스, 맥OS
VirtualPC	윈도 계열 OS	윈도 계열OS, 일부 리눅스, 솔라리스
VirtualBox	윈도 계열, 대부분의 리눅스, 맥OS, 솔라리스	윈도 계열OS, 대부분의 리눅스, 솔라리스, 맥OS, OpenBSD

ISO 이미지 다운로드 (ubuntu-24.04.4-desktop-amd64.iso)

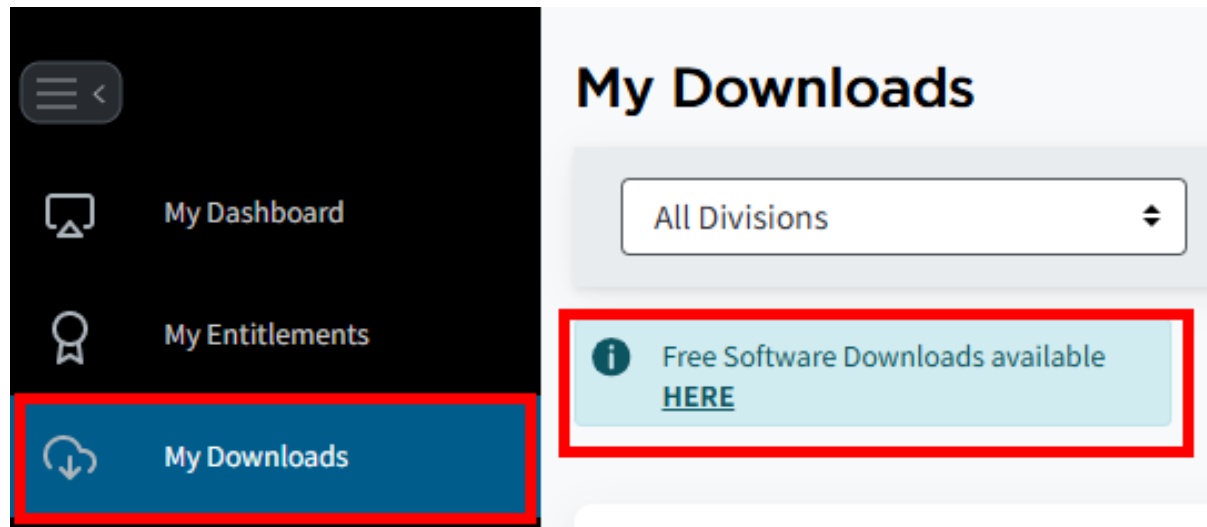
- <https://ubuntu.com/download/desktop>



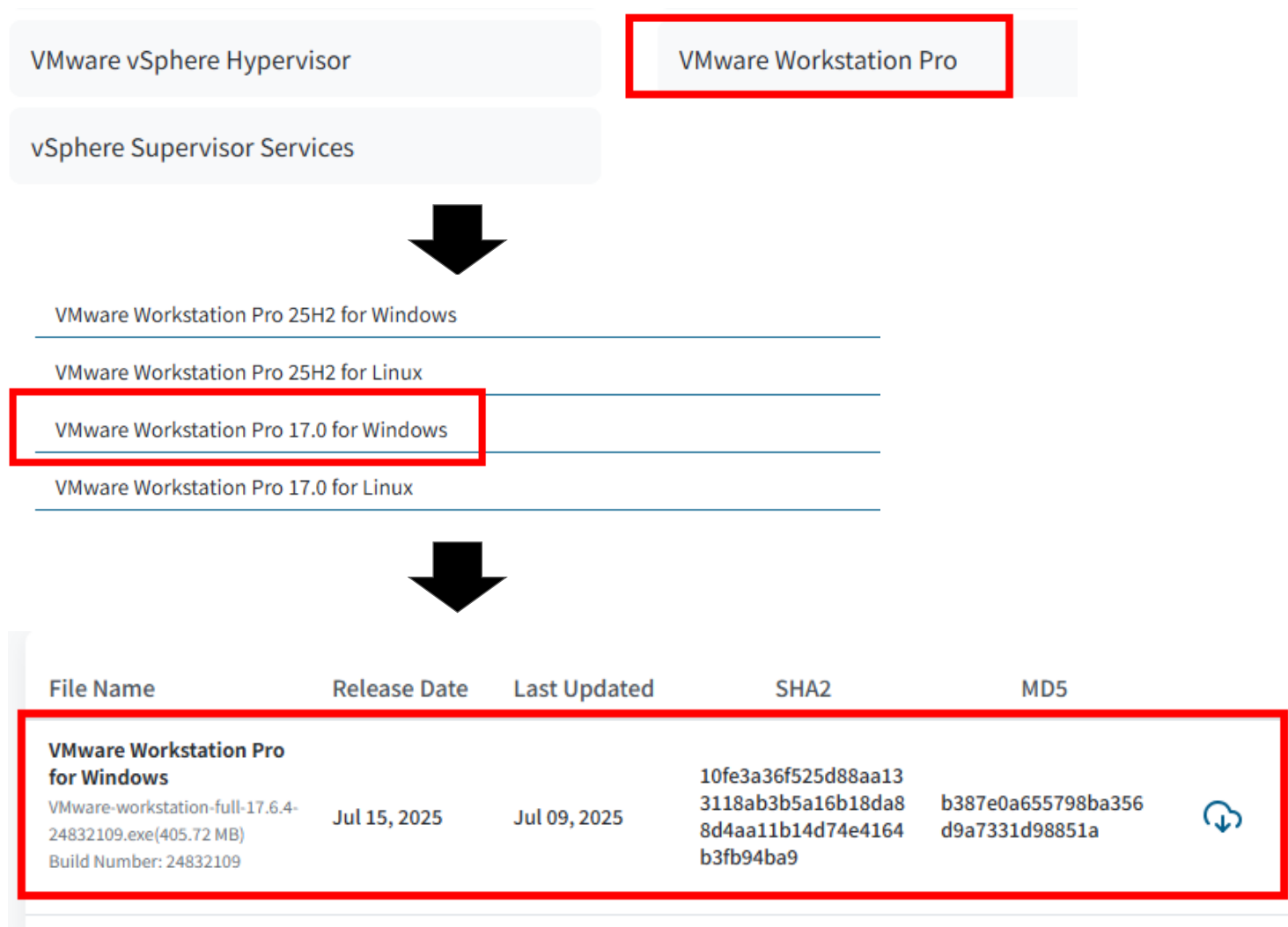
1.1 실습 환경 구성

VMWare 설치

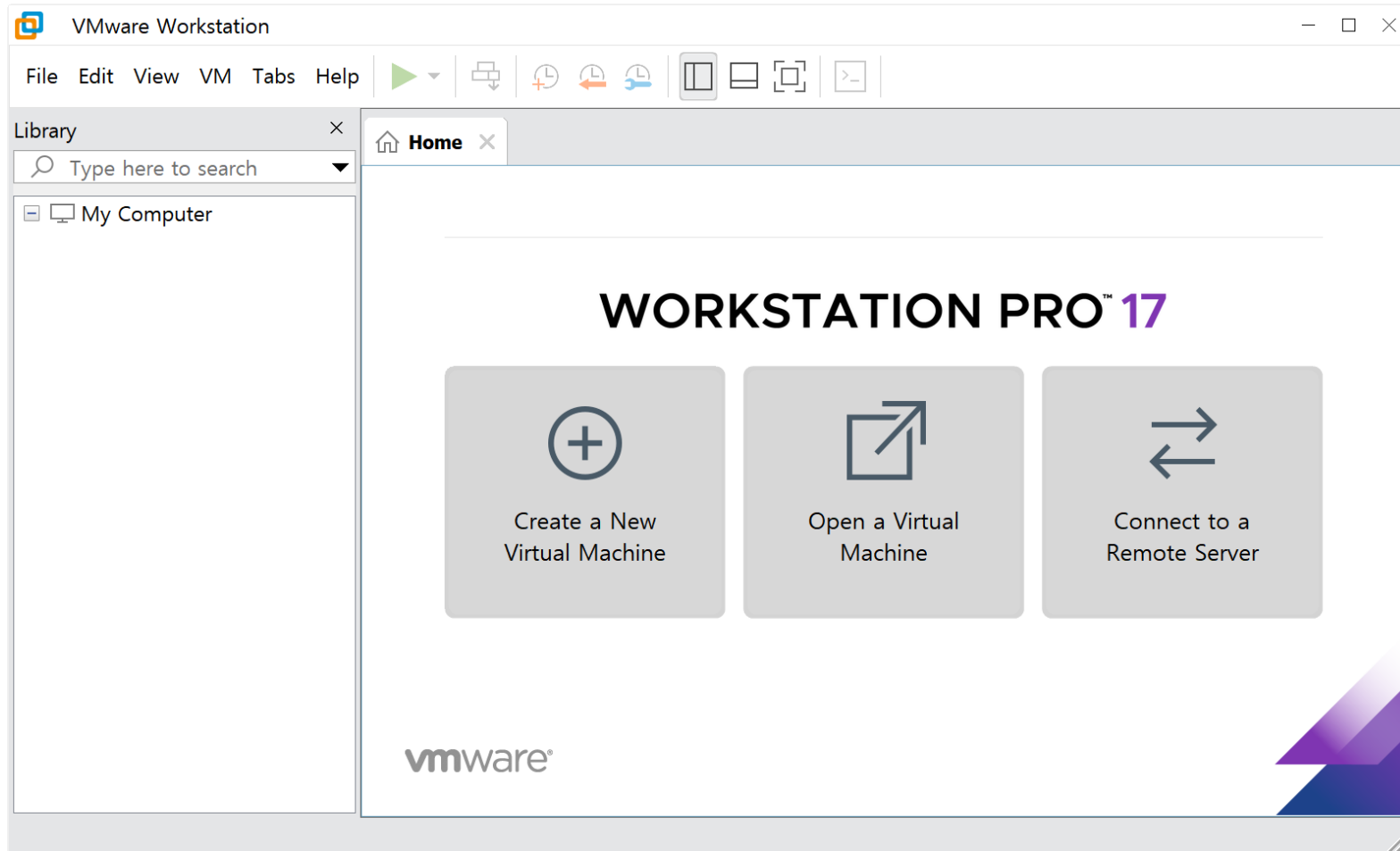
- <https://profile.broadcom.com/web/registration>
- <https://support.broadcom.com/group/ecx/free-downloads>



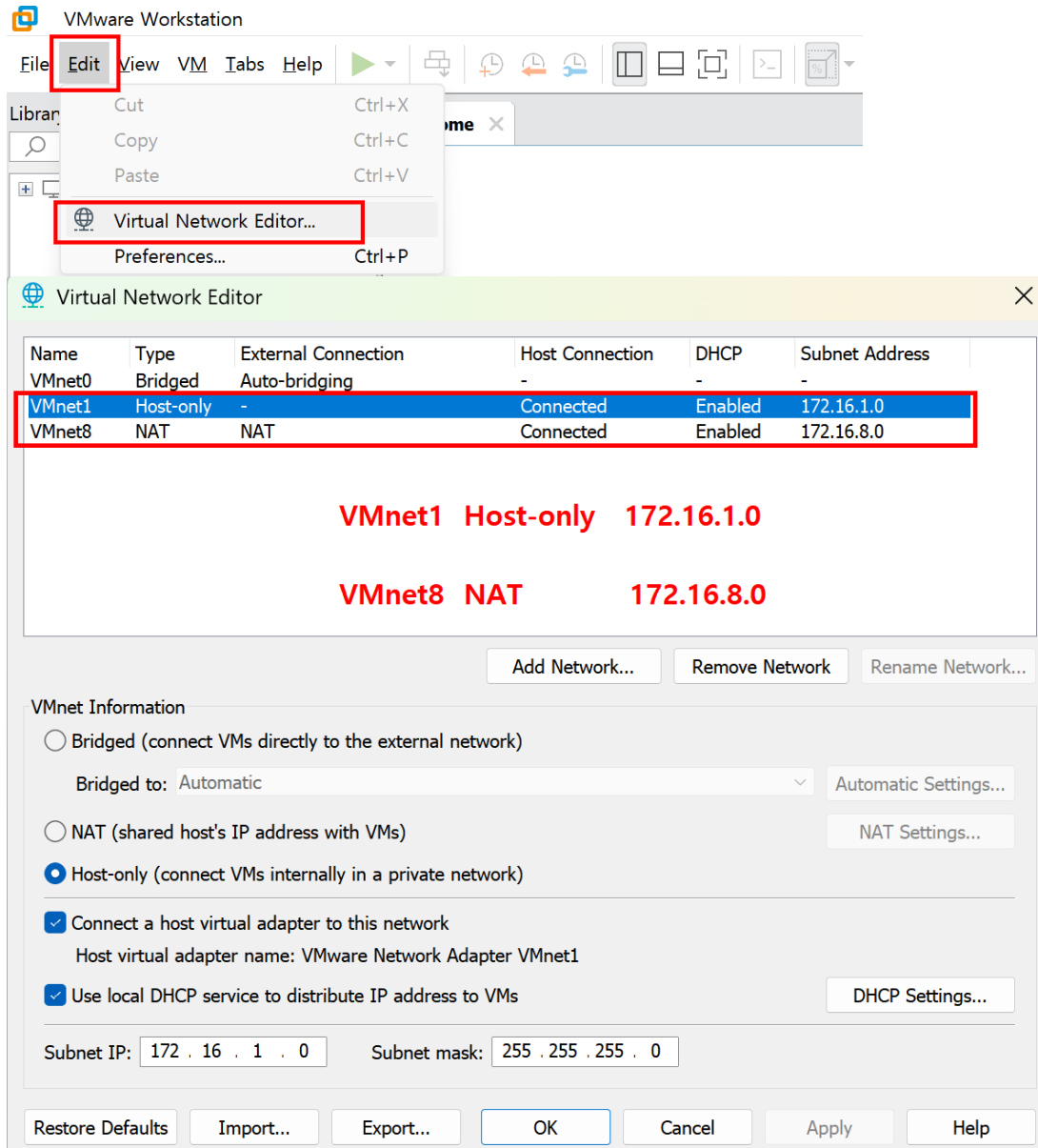
1.1 실습 환경 구성



1.1 실습 환경 구성



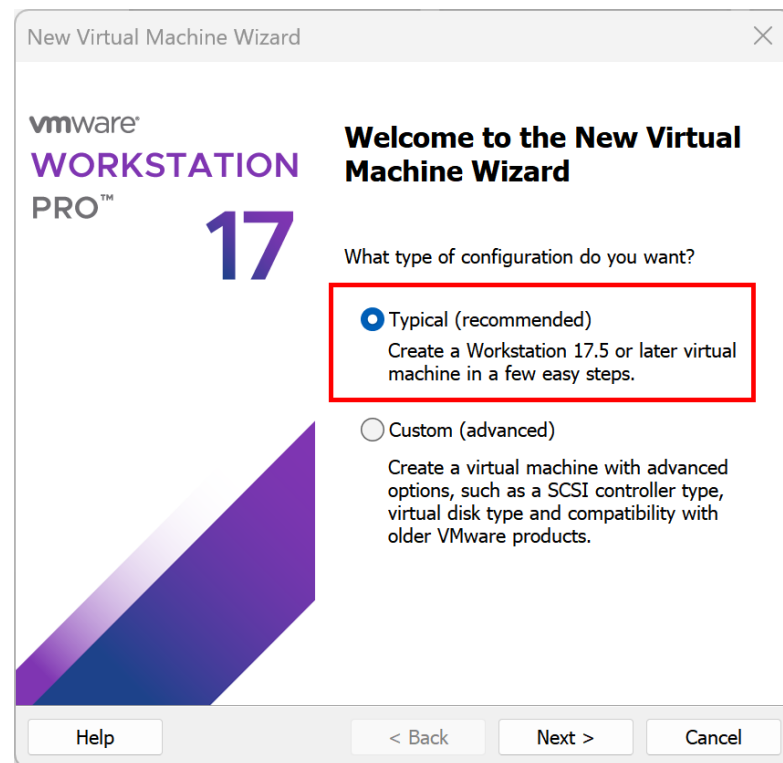
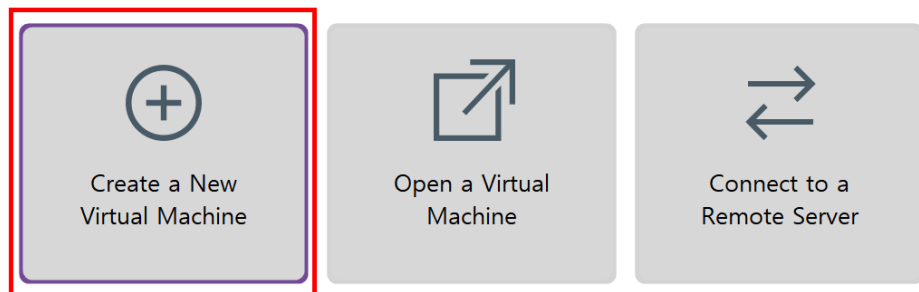
1.1 실습 환경 구성



1.1 실습 환경 구성

VMWare 에 Ubuntu 설치

WORKSTATION PRO™ 17



1.1 실습 환경 구성

VMWare 에 Ubuntu 설치

New Virtual Machine Wizard

Guest Operating System Installation
A virtual machine is like a physical computer; it needs an operating system. How will you install the guest operating system?

Install from:

☐ Installer disc:
No drives available

☒ Installer disc image file (iso): **다운받은 ubuntu ISO 이미지 파일 선택**
C:\Users\User\Downloads\ubuntu-24.04.4-desktop-a Browse...

☐ Ubuntu 64-bit 24.04.4 detected.
This operating system will use Easy Install. [\(What's this?\)](#)

☐ I will install the operating system later.
The virtual machine will be created with a blank hard disk.

Help < Back Next > Cancel



New Virtual Machine Wizard

Easy Install Information
This is used to install Ubuntu 64-bit.

Personalize Linux

Full name: user1 **user1**

User name: user1 **user1**

Password: **1234**

Confirm: **1234**

Help < Back Next > Cancel

1.1 실습 환경 구성

VMWare 에 Ubuntu 설치

New Virtual Machine Wizard

Name the Virtual Machine
What name would you like to use for this virtual machine?

Virtual machine name:
Ubuntu 64-bit

Location: **c:\vmware_ubuntu64**
C:\vmware_ubuntu64 Browse...

The default location can be changed at Edit > Preferences.

< Back Next > Cancel



New Virtual Machine Wizard

Specify Disk Capacity
How large do you want this disk to be?

The virtual machine's hard disk is stored as one or more files on the host computer's physical disk. These file(s) start small and become larger as you add applications, files, and data to your virtual machine.

Maximum disk size (GB): 20.0

Recommended size for Ubuntu 64-bit: 20 GB

☐ Store virtual disk as a single file

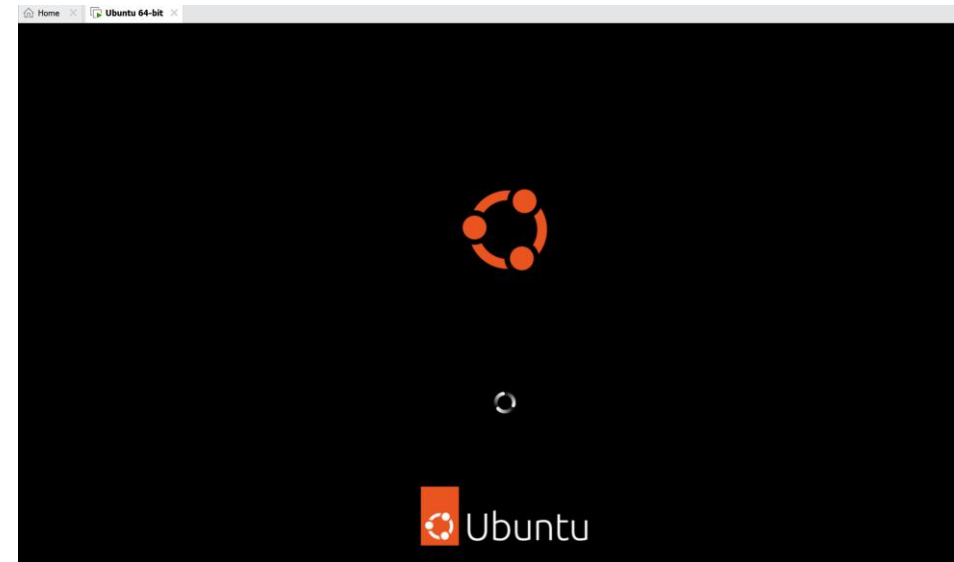
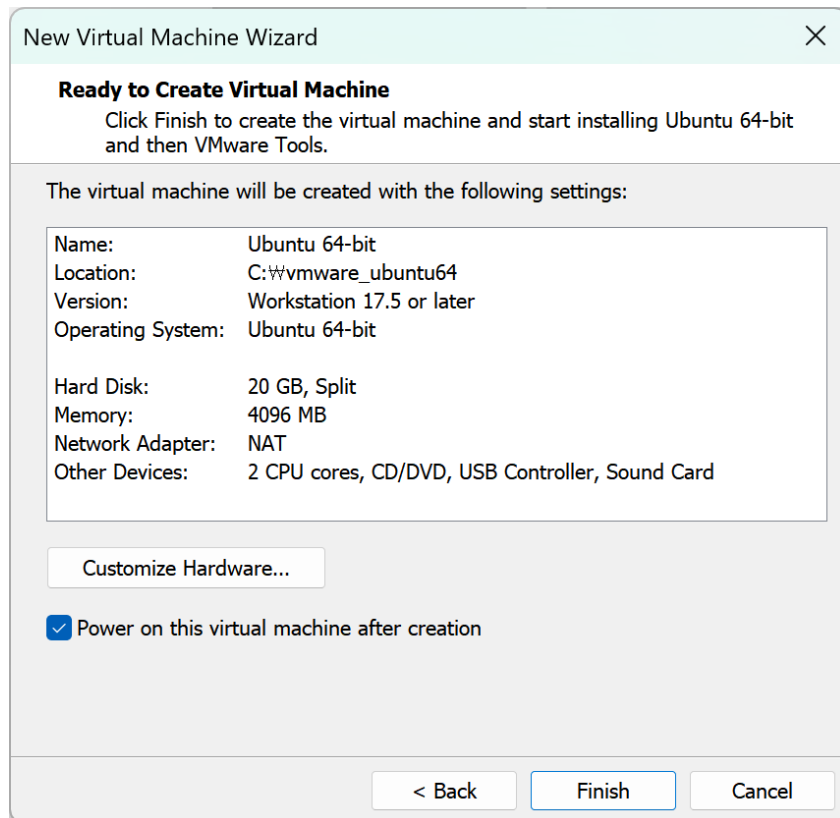
☒ Split virtual disk into multiple files

Splitting the disk makes it easier to move the virtual machine to another computer but may reduce performance with very large disks.

Help < Back Next > Cancel

1.1 실습 환경 구성

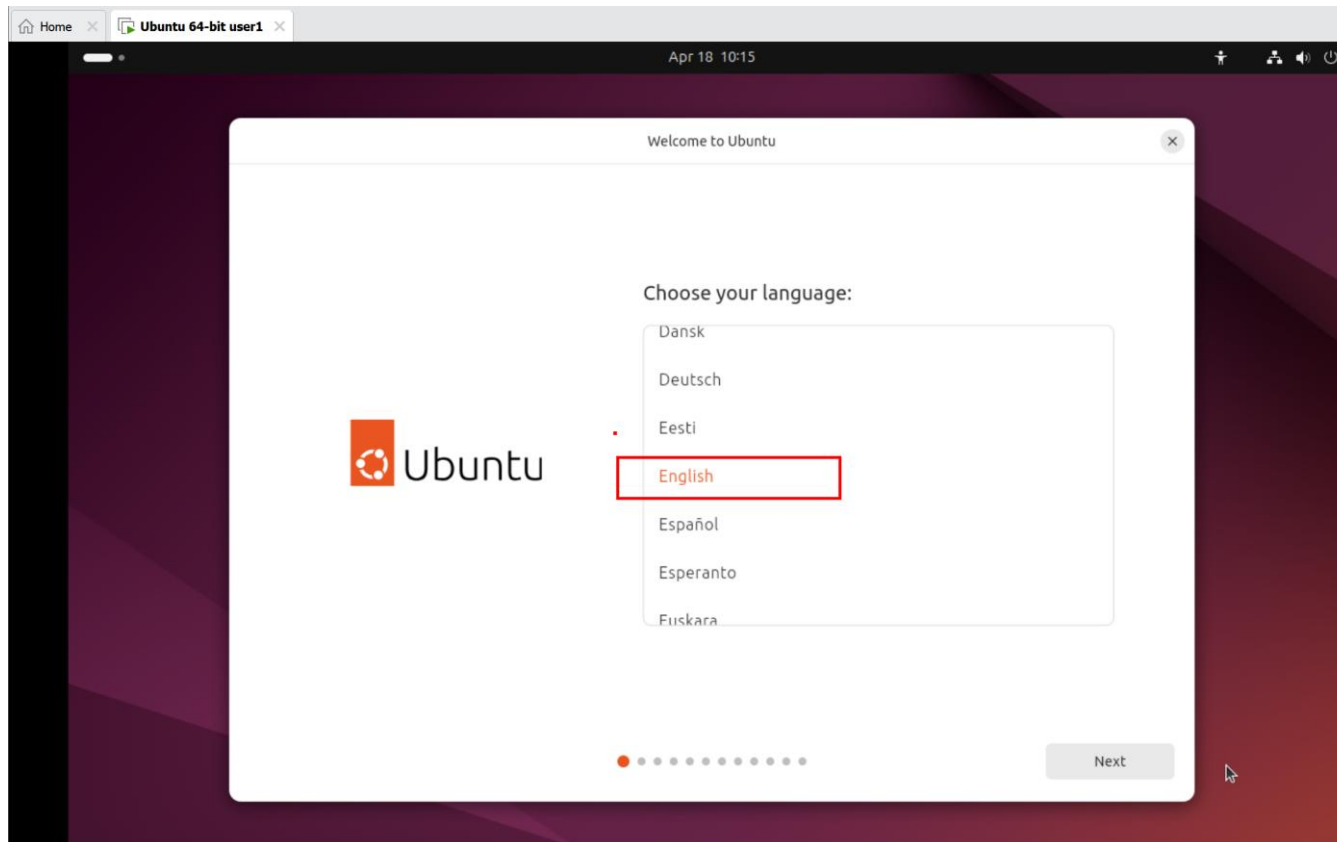
VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

1. 실습환경 구성

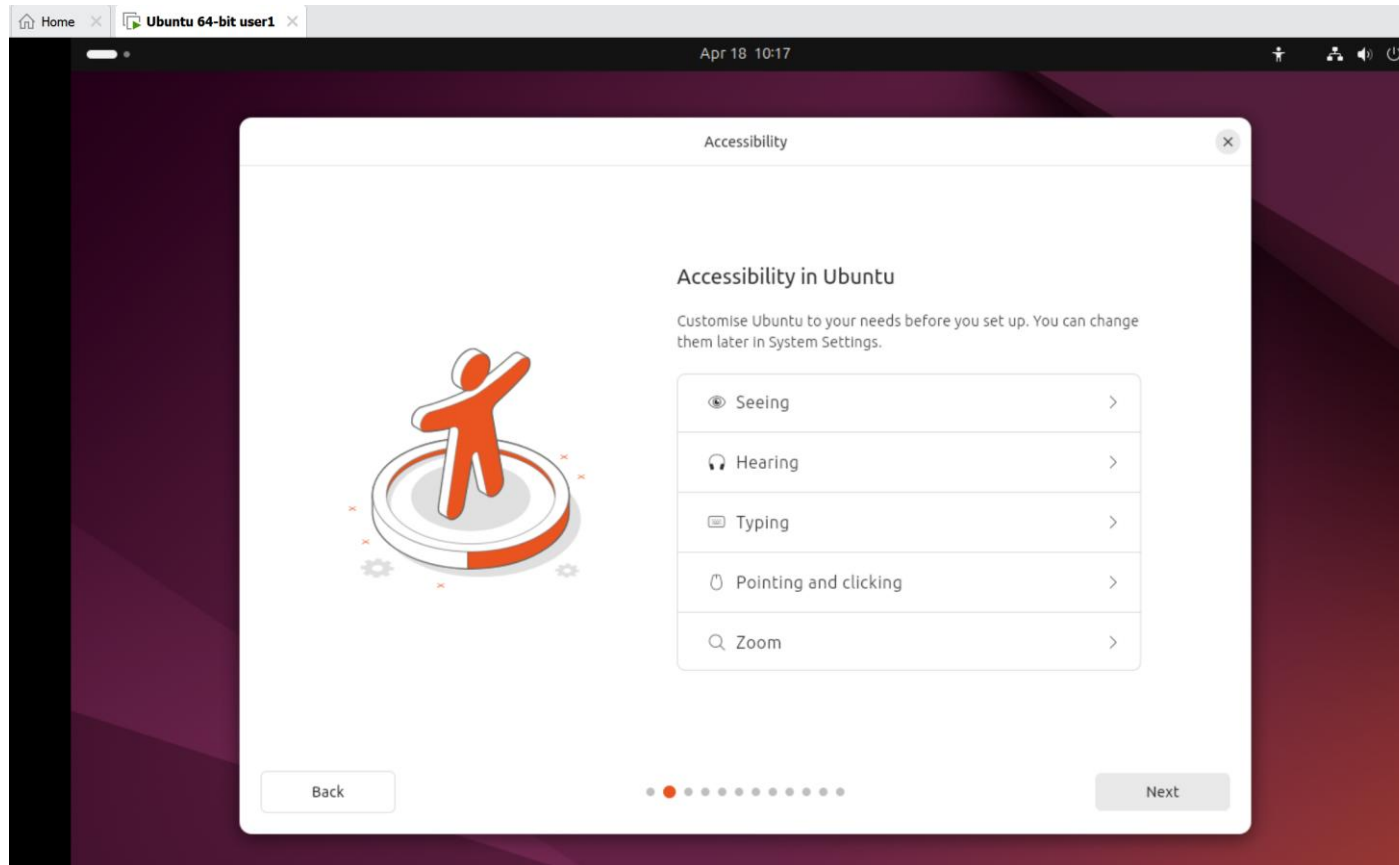
VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

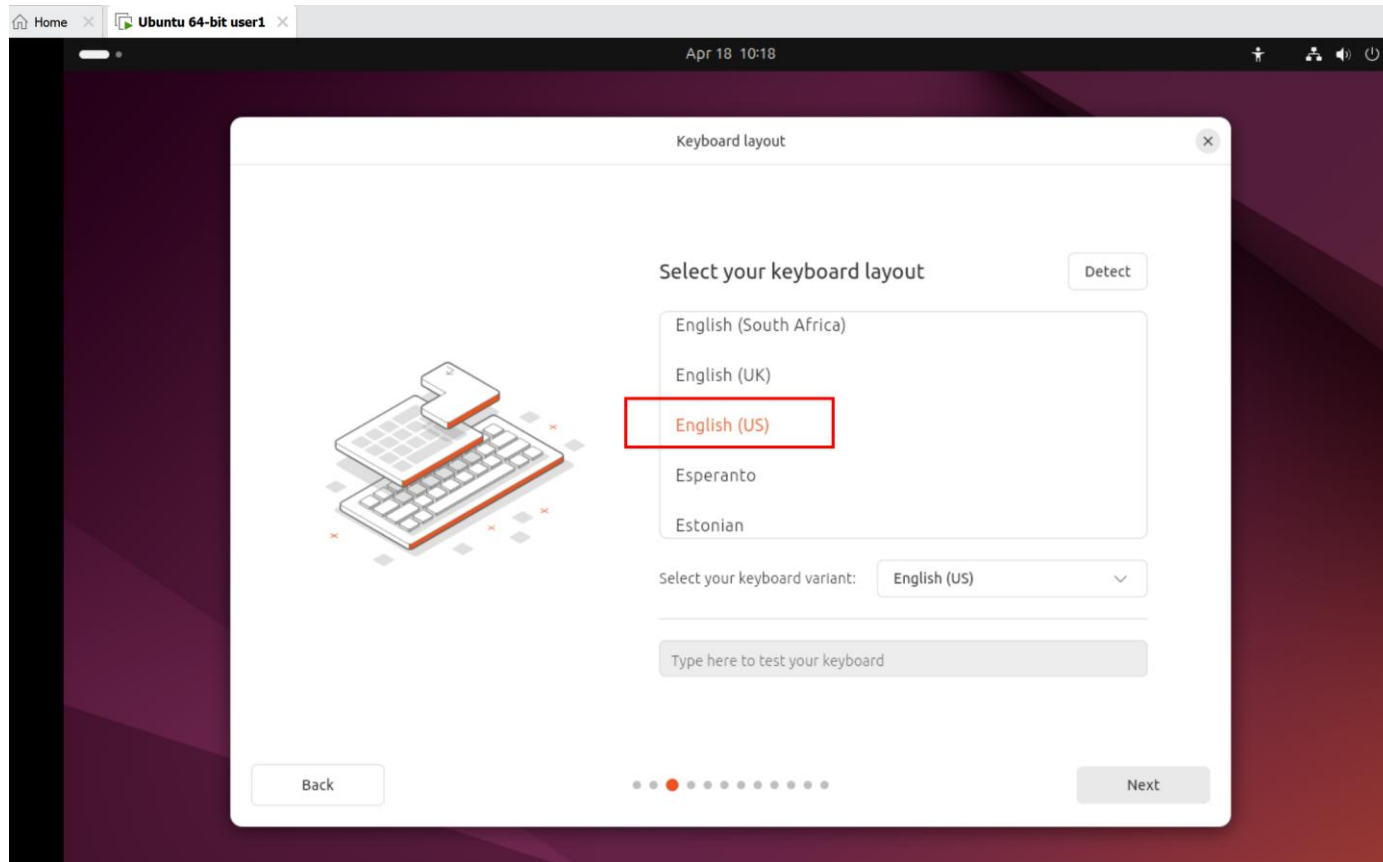
1. 실습환경 구성

VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

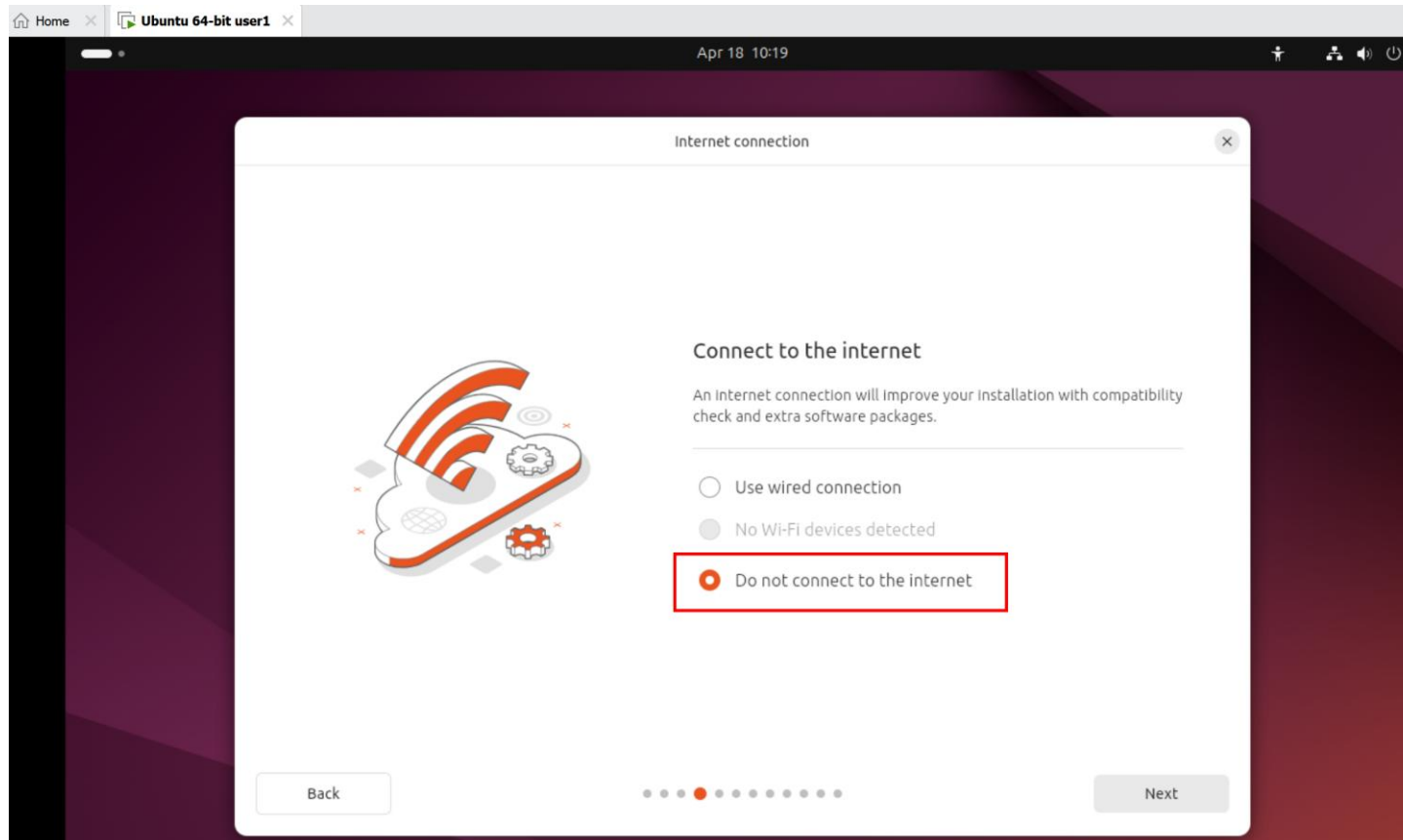
VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

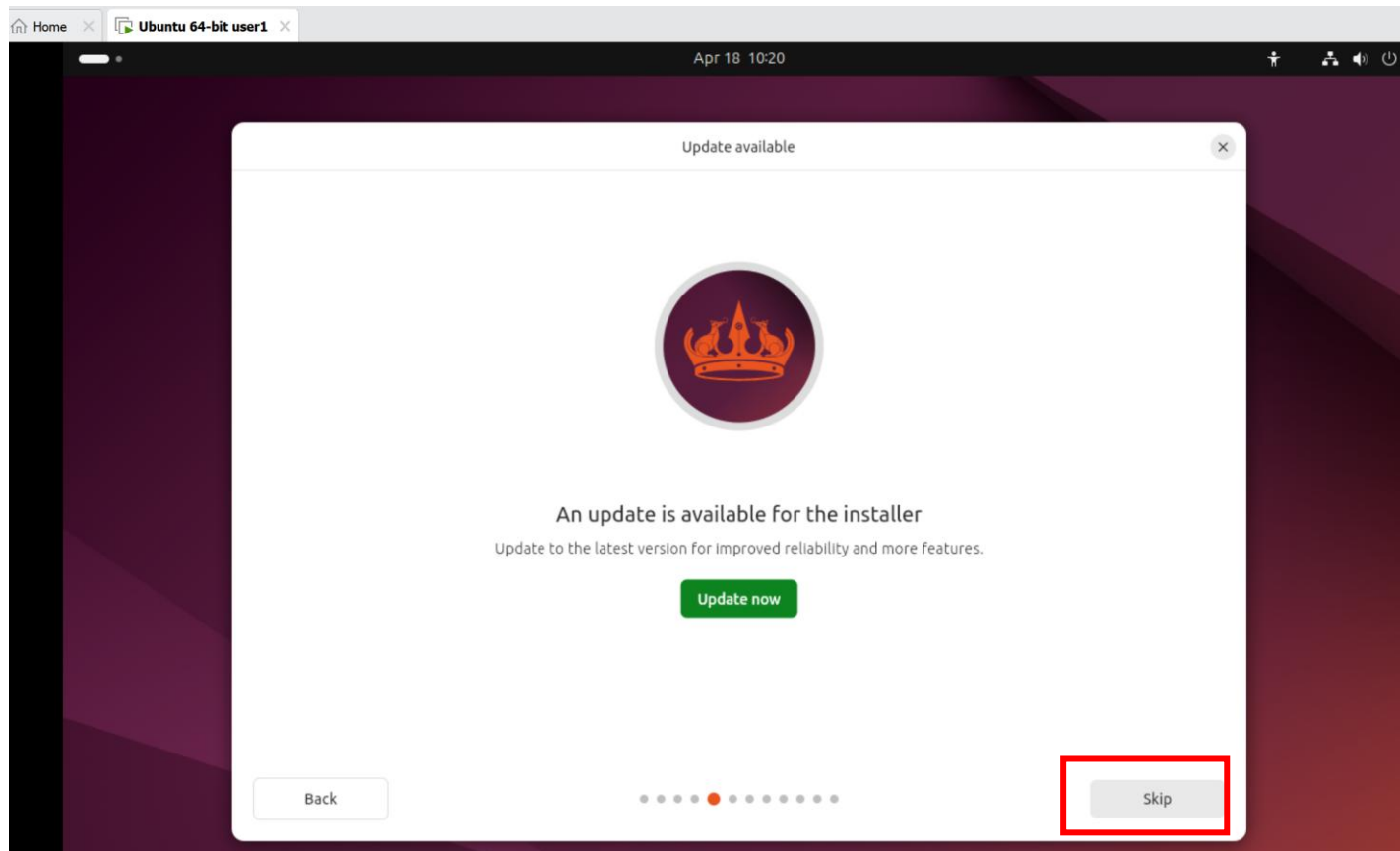
1. 실습환경 구성

VMWare 에 Ubuntu 설치



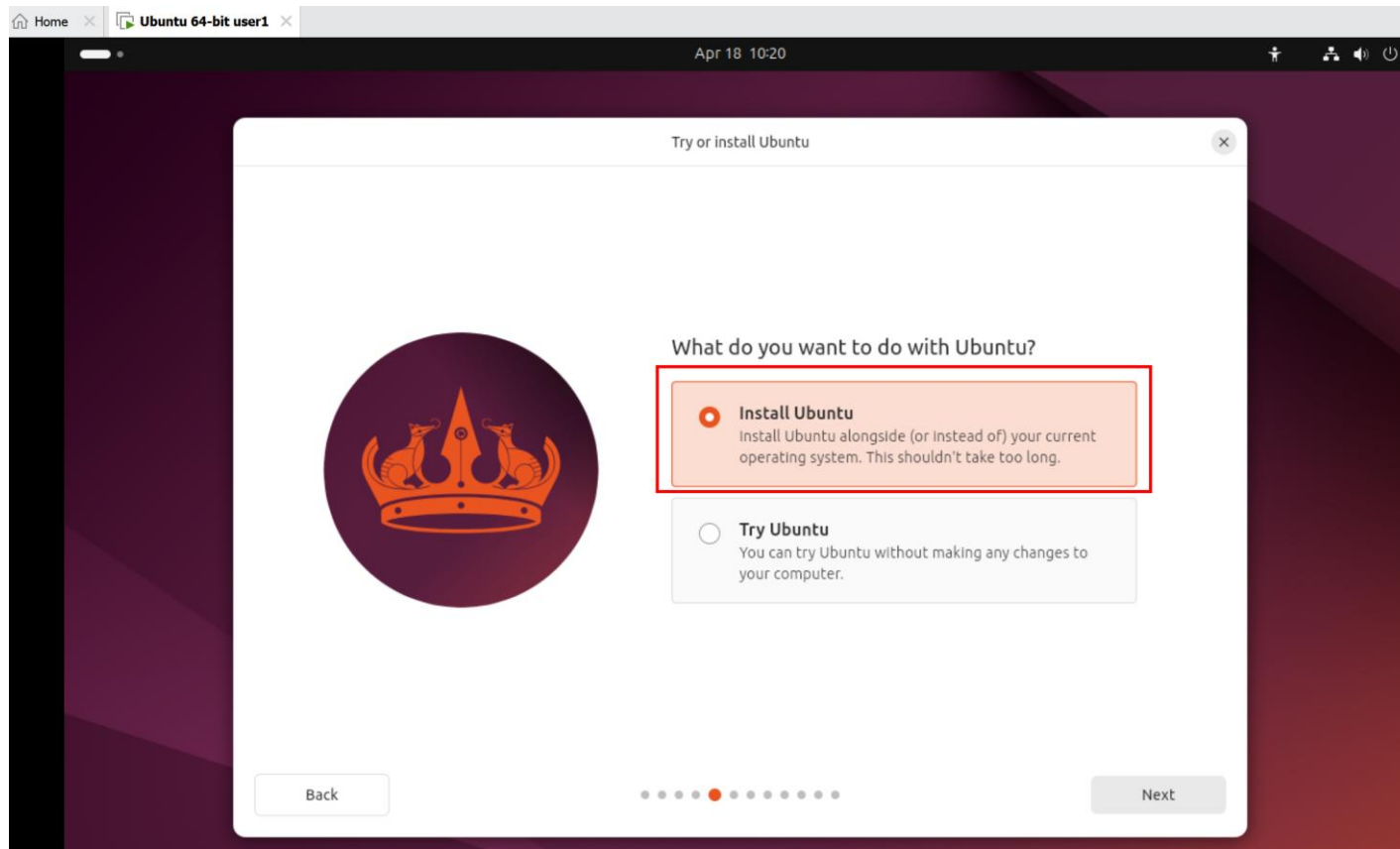
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치



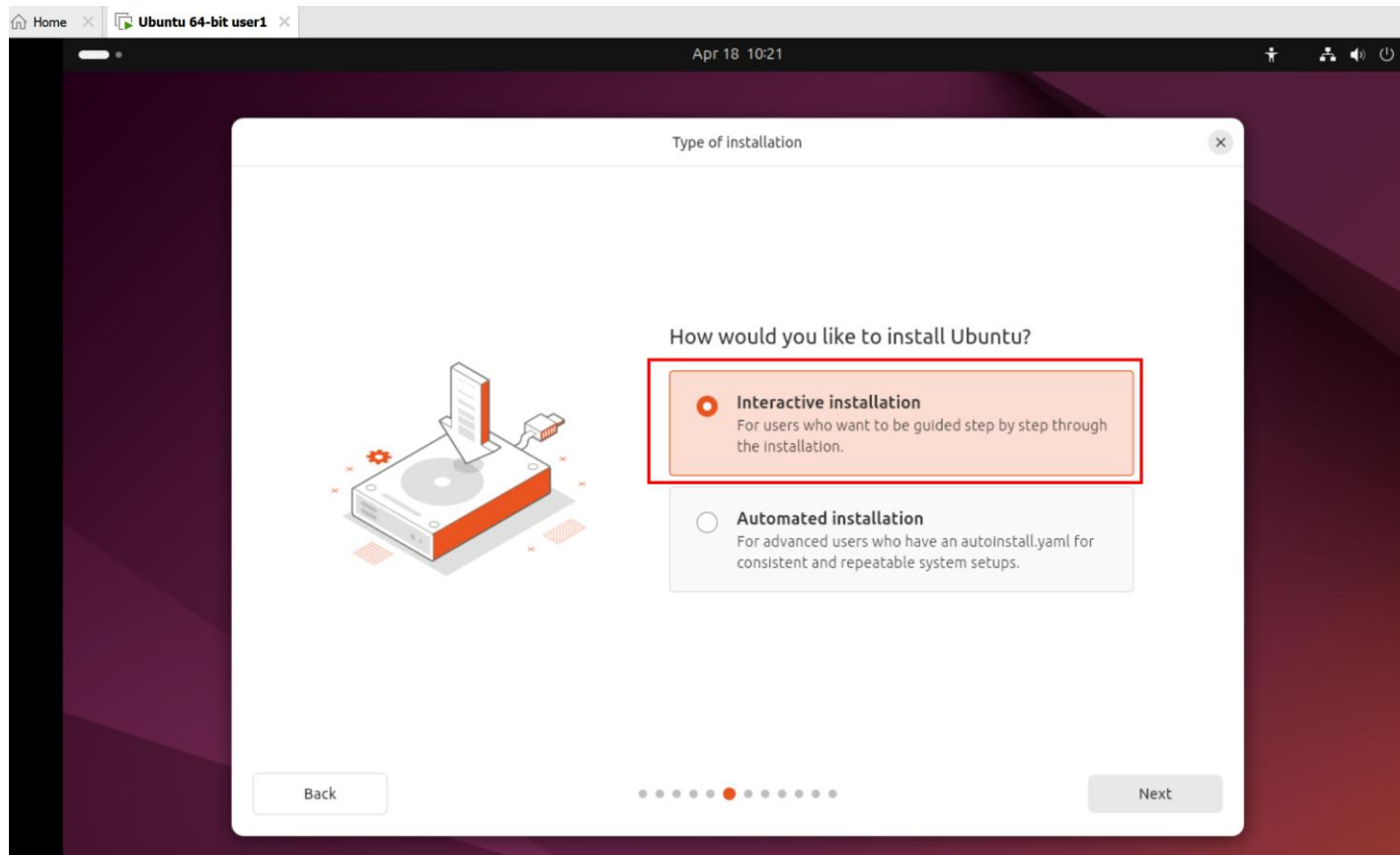
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치



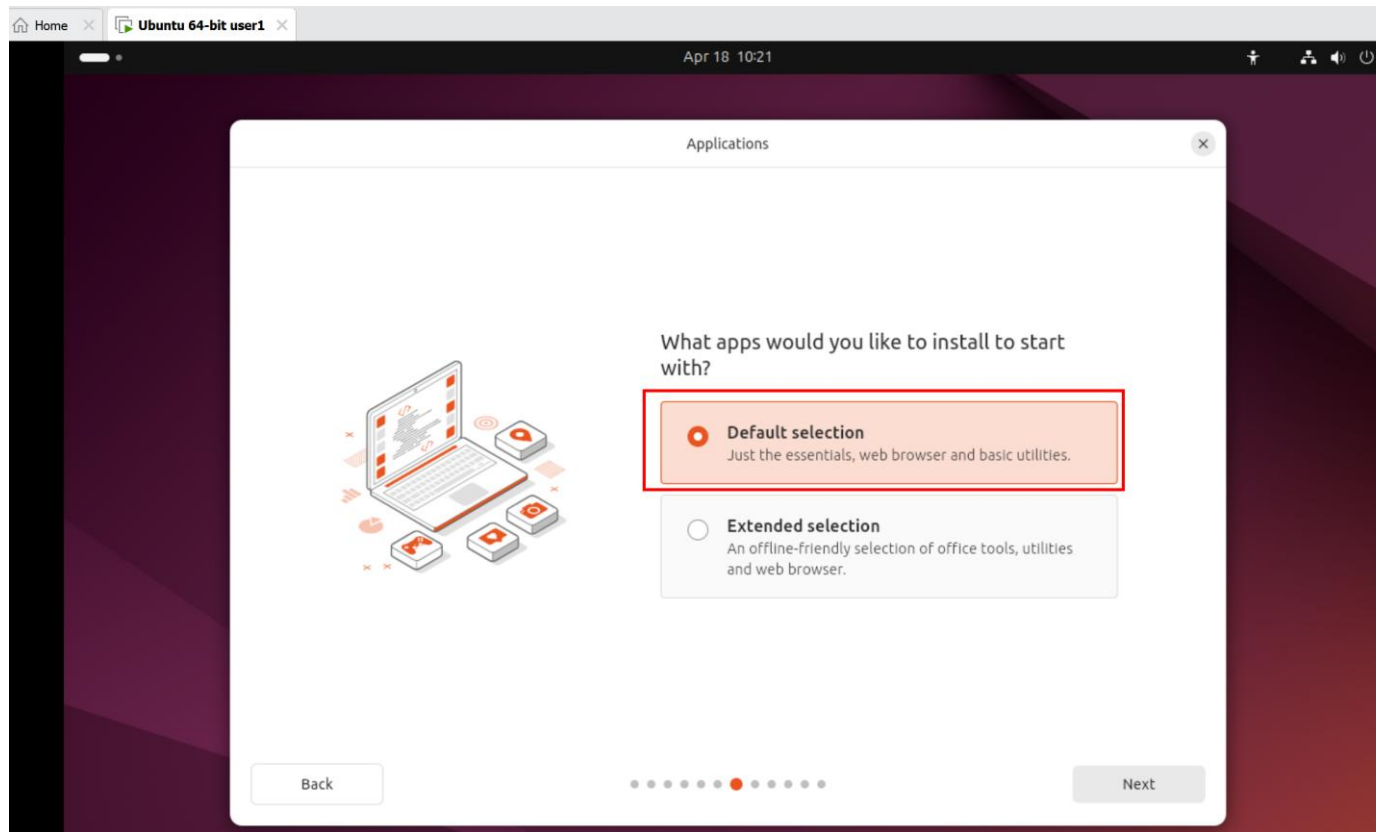
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

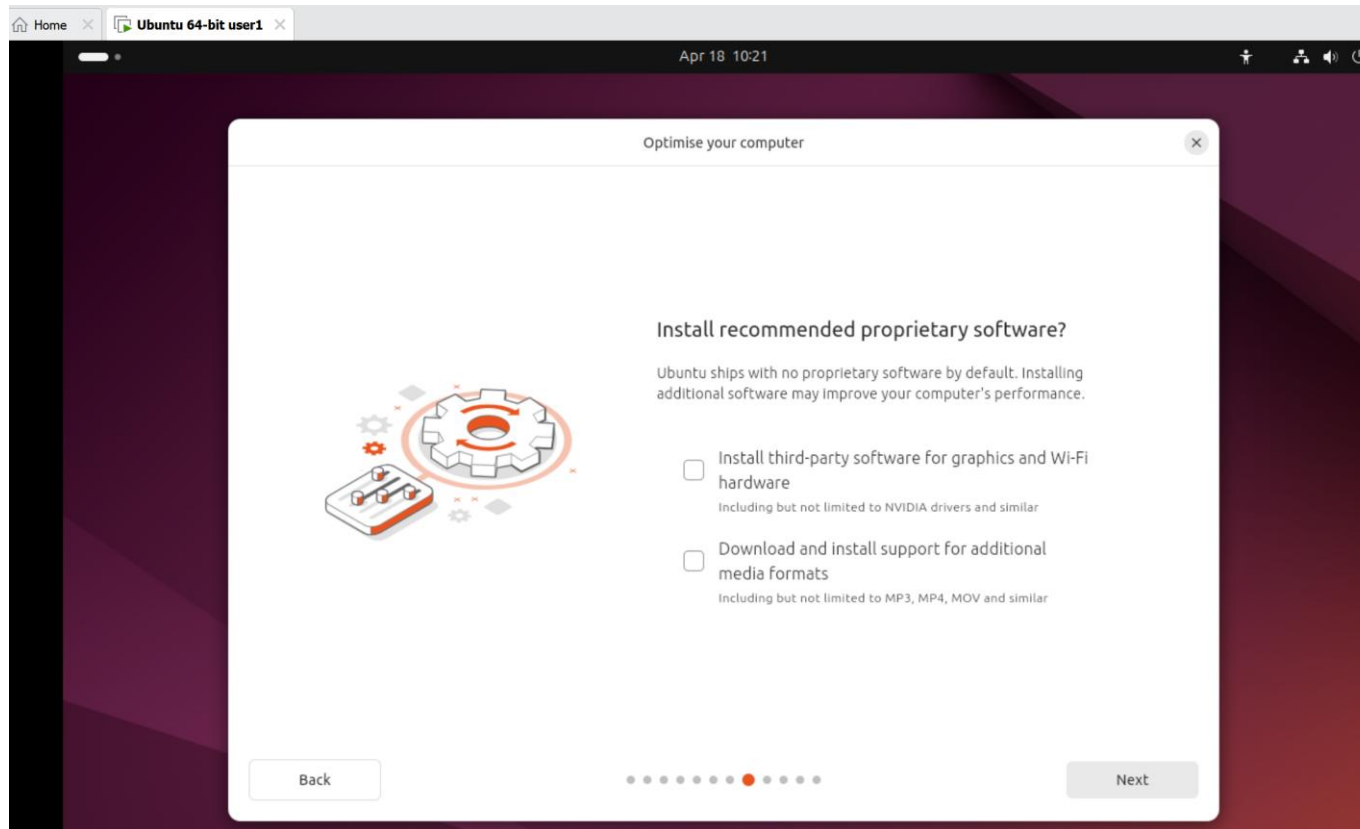
VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

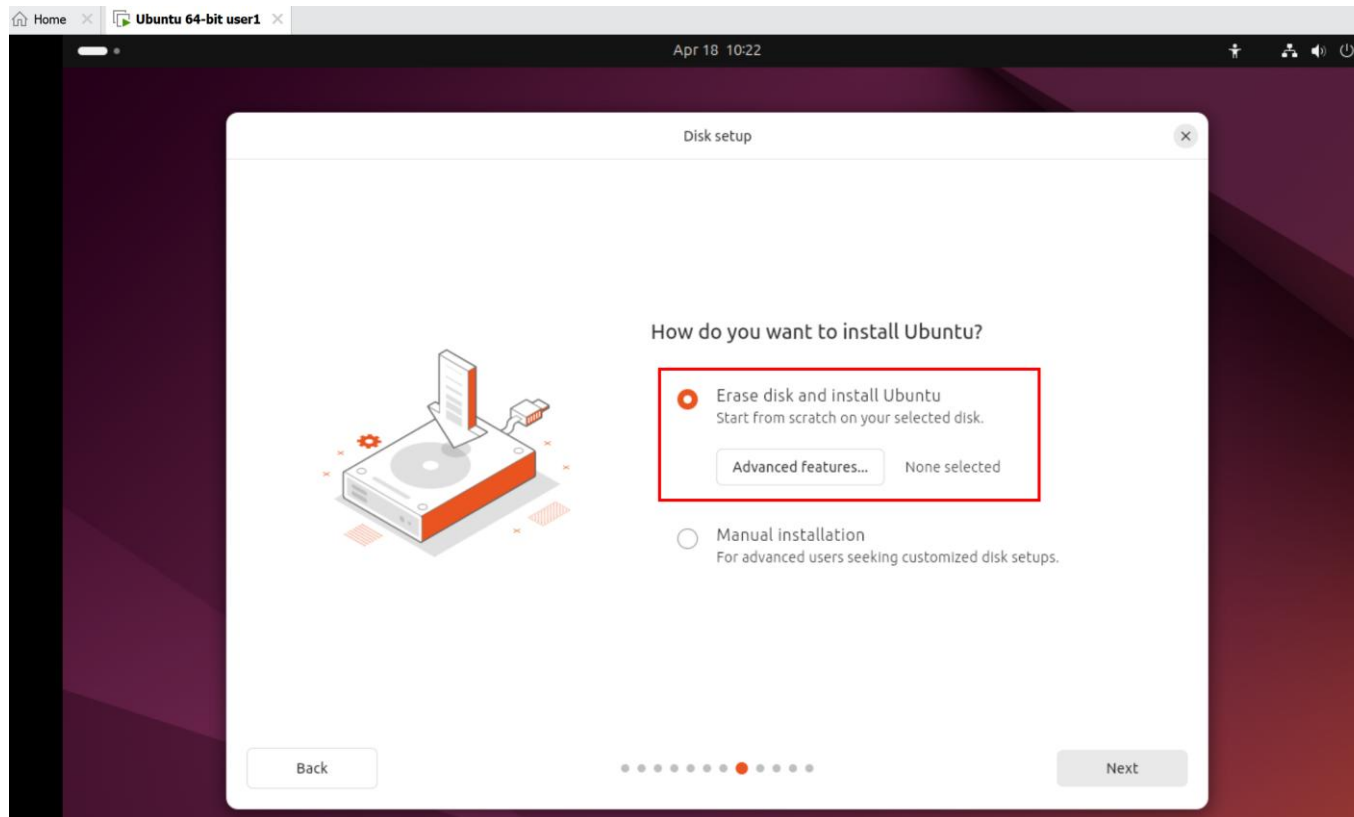
1. 실습환경 구성

VMWare 에 Ubuntu 설치



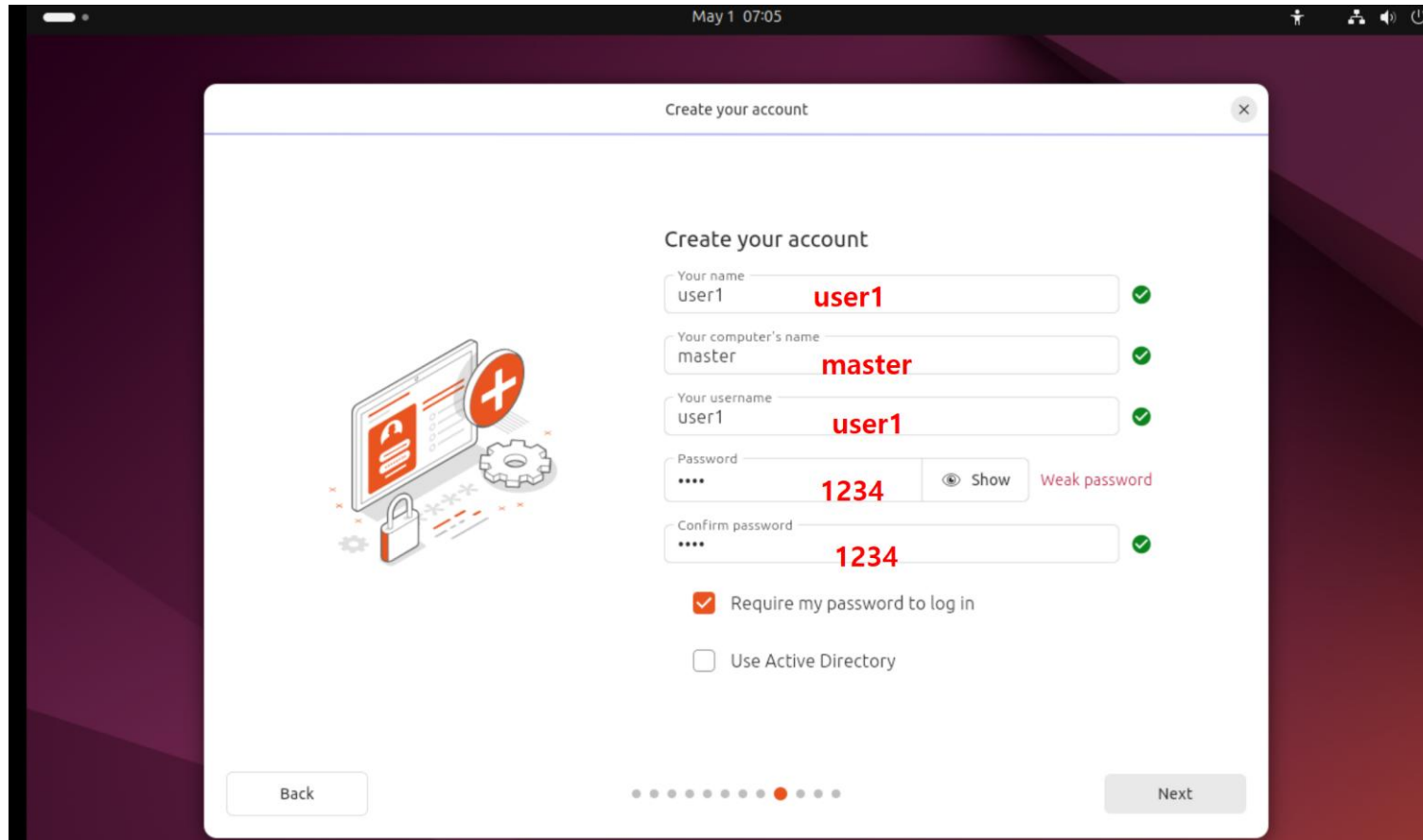
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

VMWare 에 Ubuntu 설치 (user1 계정 생성)



May 1 07:05

Create your account

Create your account

Your name
user1 user1 ✓

Your computer's name
master master ✓

Your username
user1 user1 ✓

Password
**** 1234 Show Weak password

Confirm password
**** 1234 ✓

☒ Require my password to log in

☐ Use Active Directory

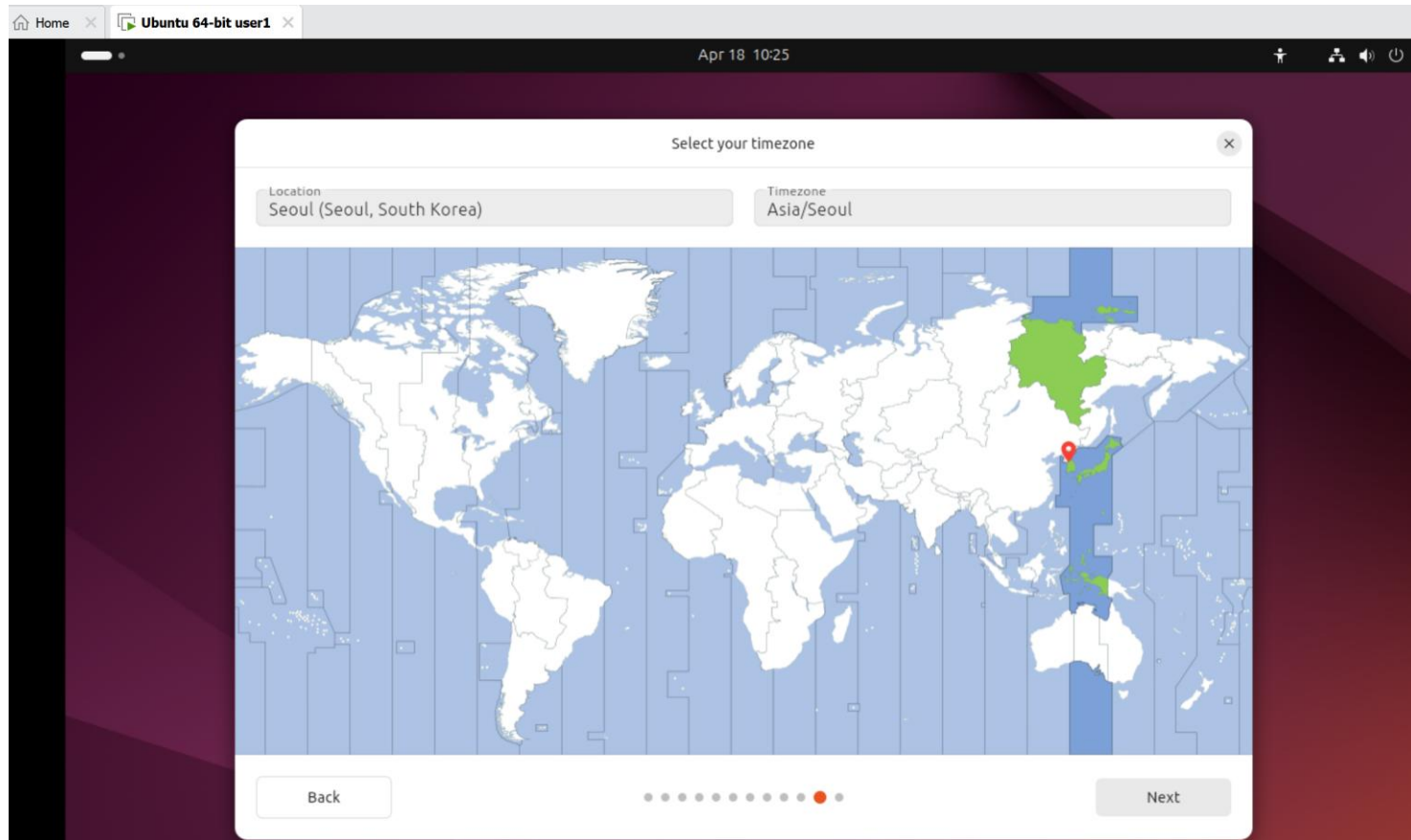
Back

Next

1.1 실습 환경 구성

1. 실습환경 구성

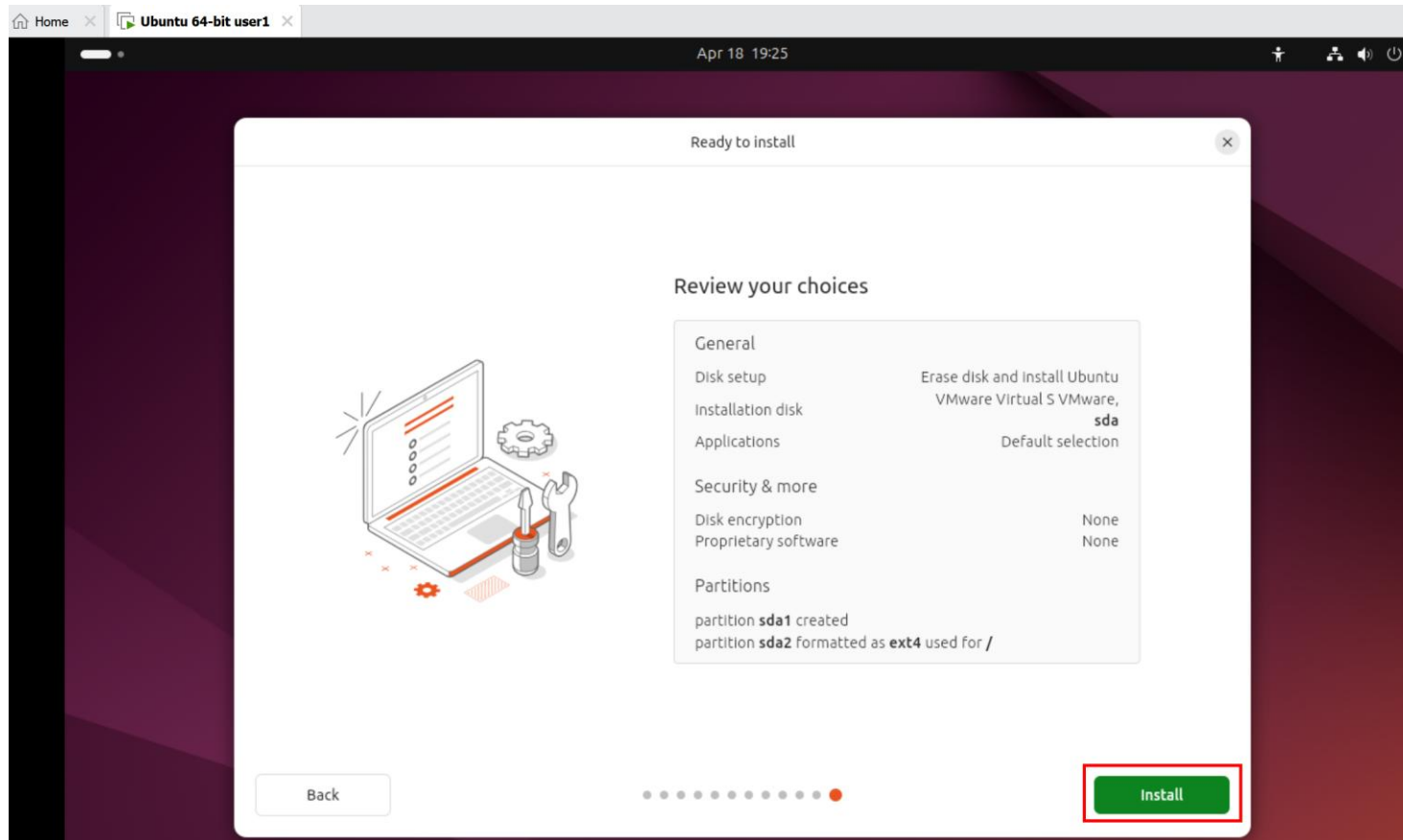
VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

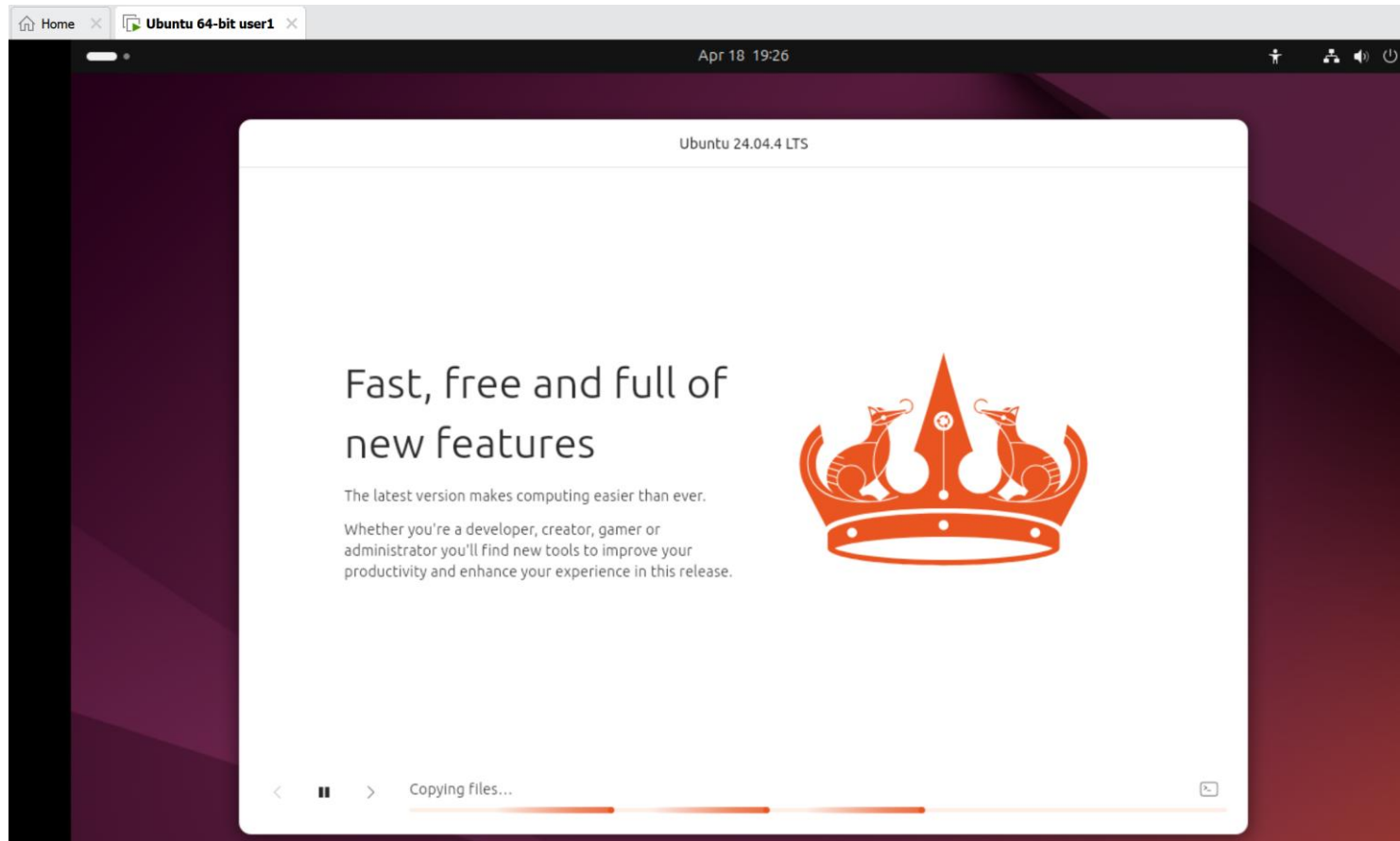
1. 실습환경 구성

VMWare 에 Ubuntu 설치



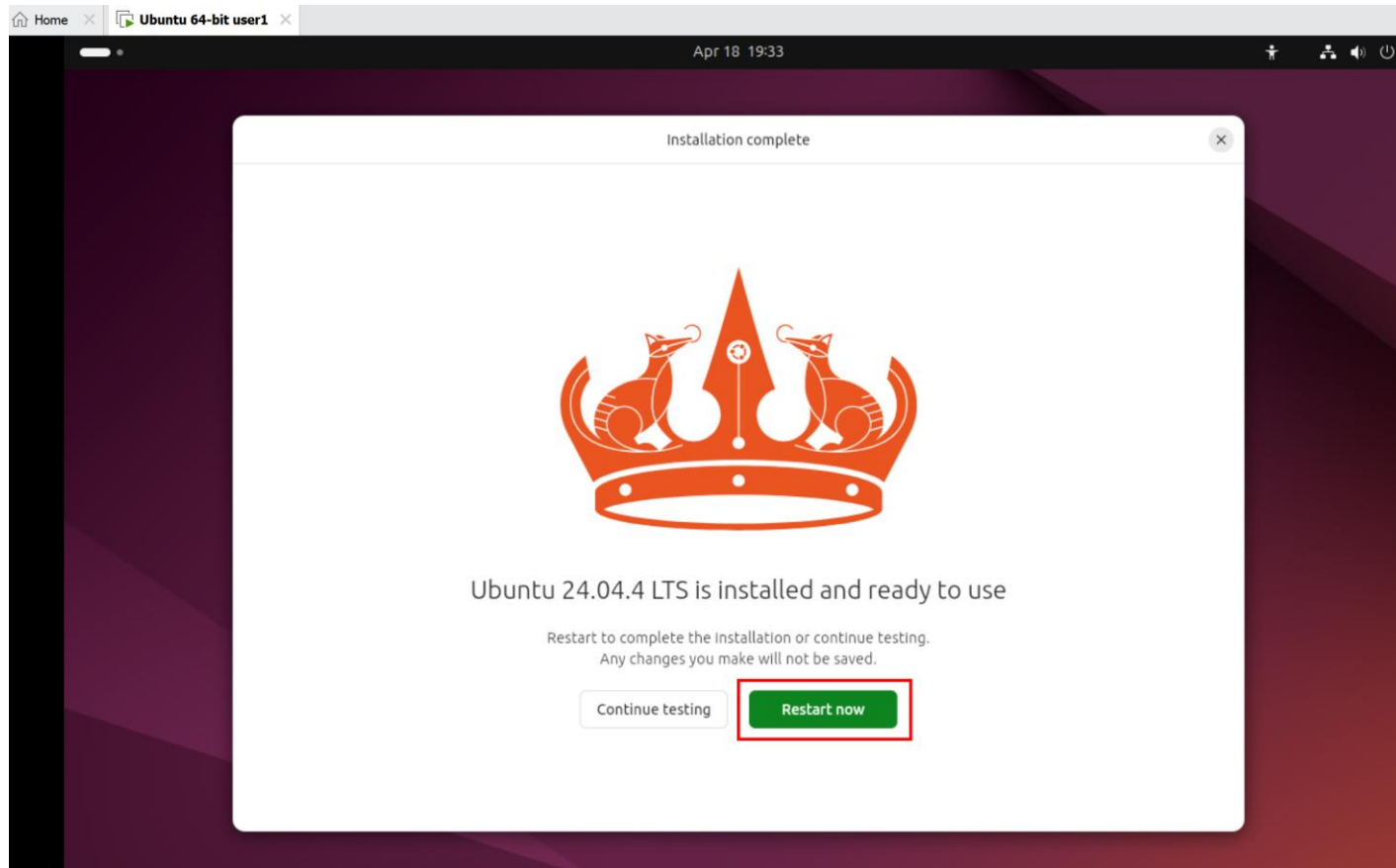
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치



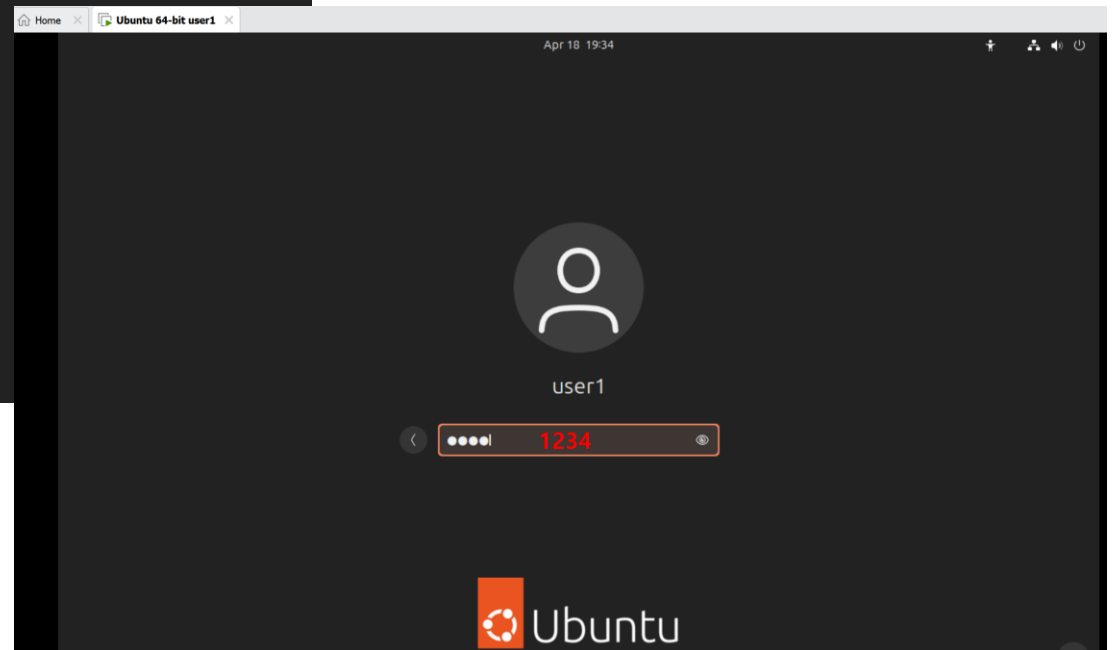
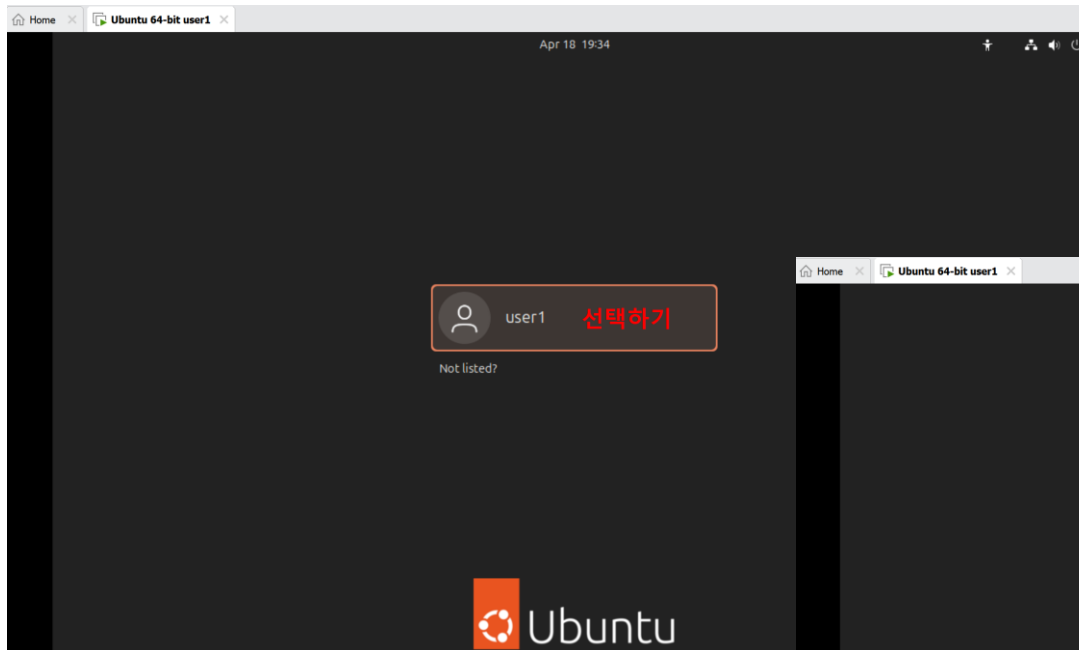
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치 (설치 완료)



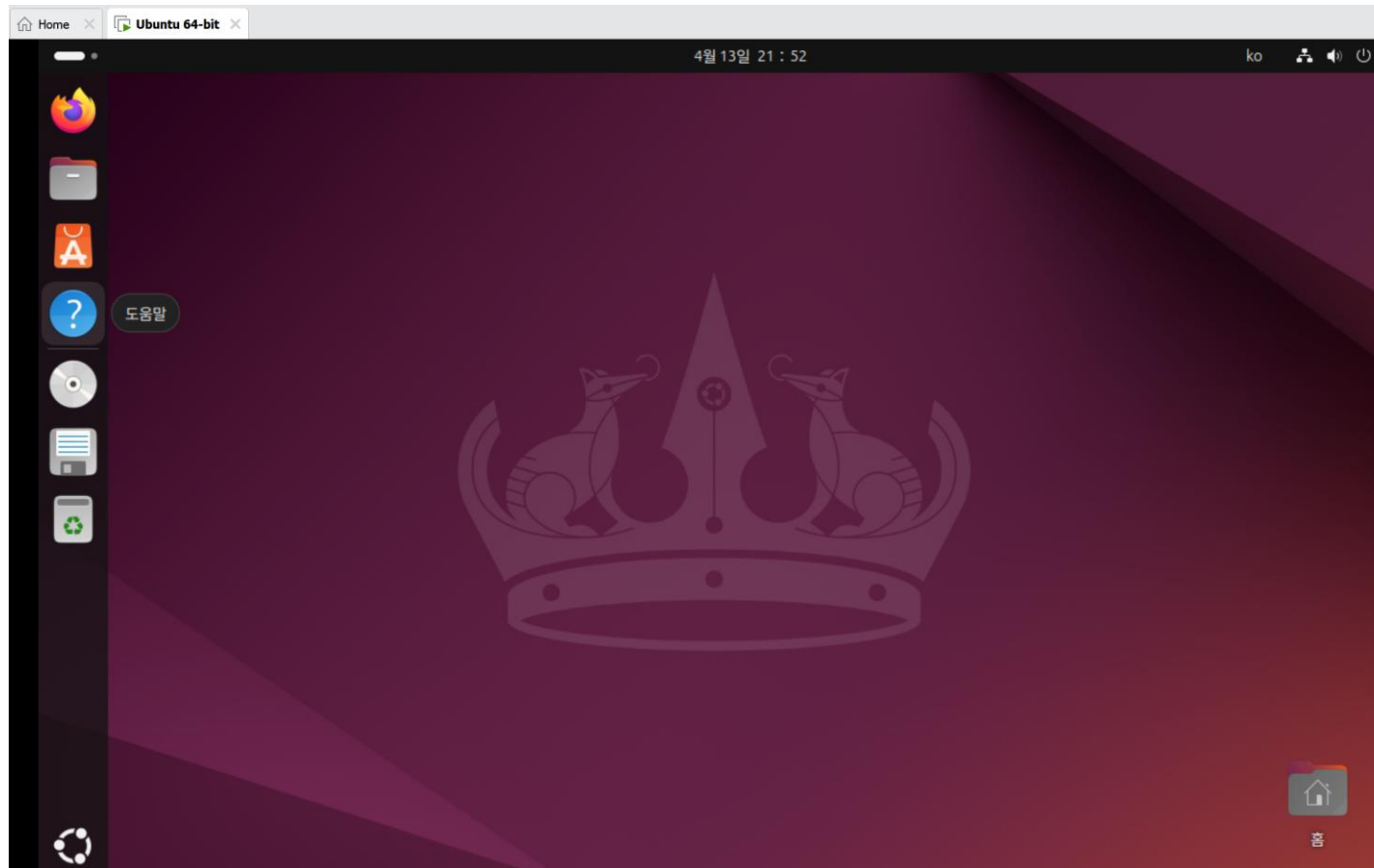
1.1 실습 환경 구성

VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

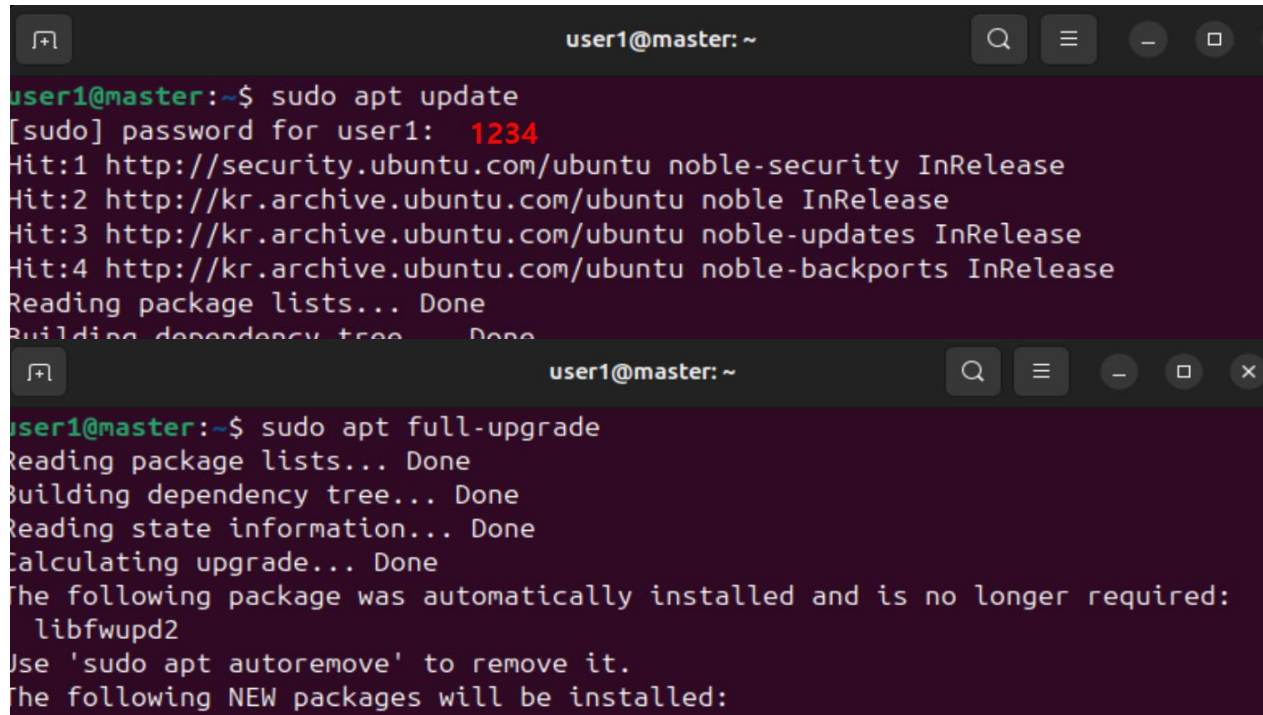
VMWare 에 Ubuntu 설치



1.1 실습 환경 구성

모든 응용 프로그램 업데이트

- 터미널 열기
- `sudo apt update`
- `sudo apt full-upgrade`

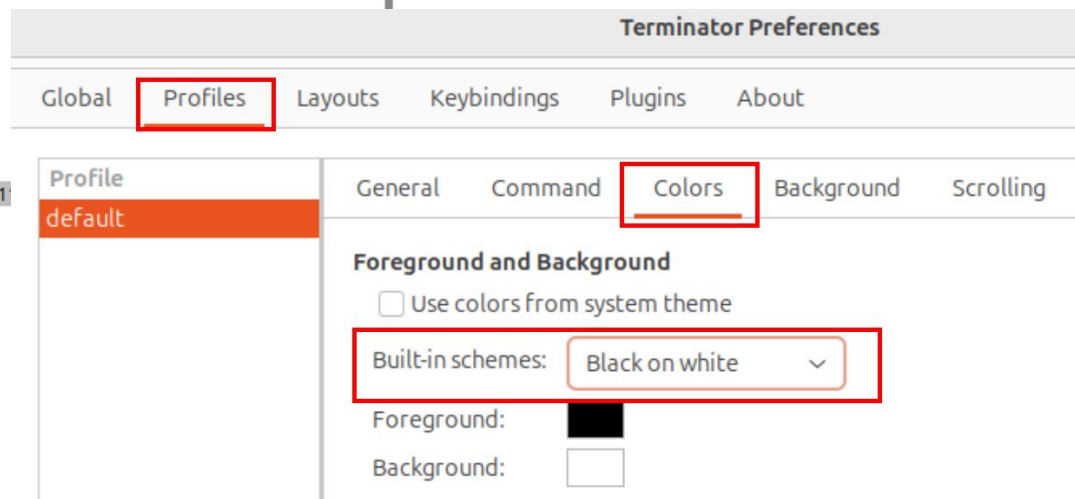
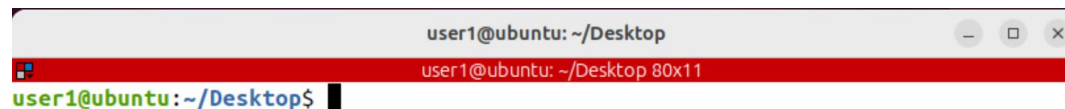


```
user1@master: ~  
user1@master:~$ sudo apt update  
[sudo] password for user1: 1234  
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease  
Hit:2 http://kr.archive.ubuntu.com/ubuntu noble InRelease  
Hit:3 http://kr.archive.ubuntu.com/ubuntu noble-updates InRelease  
Hit:4 http://kr.archive.ubuntu.com/ubuntu noble-backports InRelease  
Reading package lists... Done  
Building dependency tree... Done  
user1@master: ~  
user1@master:~$ sudo apt full-upgrade  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Calculating upgrade... Done  
The following package was automatically installed and is no longer required:  
  libfwupd2  
Use 'sudo apt autoremove' to remove it.  
The following NEW packages will be installed:
```

1.1 실습 환경 구성

새로운 터미널 설치하기

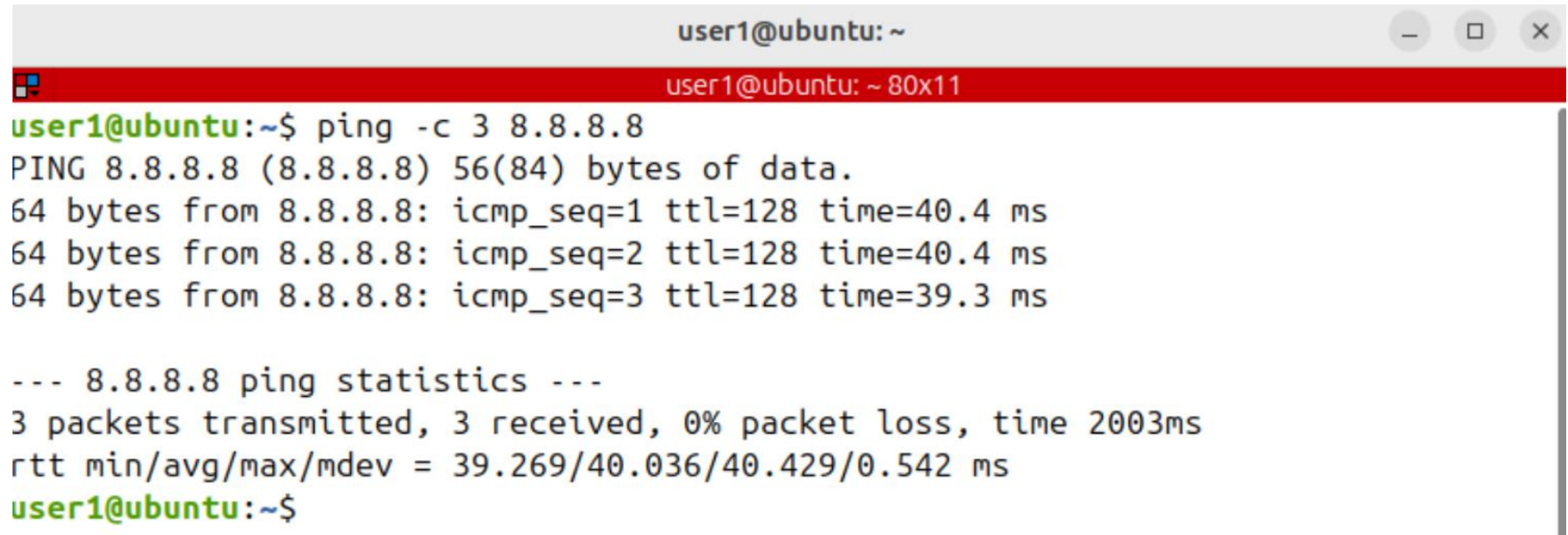
- `sudo apt update & sudo apt install terminator`



1.1 실습 환경 구성

인터넷 정상 동작 확인

- `ping -c 3 8.8.8.8`

A terminal window titled 'user1@ubuntu: ~' with standard window controls. The prompt is 'user1@ubuntu: ~ 80x11'. The user has entered the command 'ping -c 3 8.8.8.8'. The output shows three successful ping requests to 8.8.8.8 with varying response times (40.4 ms, 40.4 ms, 39.3 ms). A summary line indicates 0% packet loss and an average round-trip time of 40.036 ms.

```
user1@ubuntu: ~  
user1@ubuntu: ~ 80x11  
user1@ubuntu:~$ ping -c 3 8.8.8.8  
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=40.4 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=40.4 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=39.3 ms  
  
--- 8.8.8.8 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2003ms  
rtt min/avg/max/mdev = 39.269/40.036/40.429/0.542 ms  
user1@ubuntu:~$
```

1.1 실습 환경 구성

ubuntu에 s/w 설치

```
$ sudo -i
```

```
[sudo] password for user1: 1234
```

```
root@master:~# apt update && apt install -y curl git wget vim ssh
```

```
root@master:~# vim /etc/sudoers
```

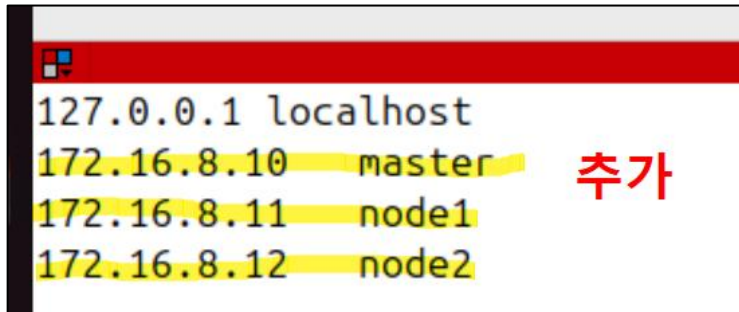
- 54행에 user1 ALL=(ALL) NOPASSWD: ALL 추가

```
53 # Allow members of group sudo to execute
54 %sudo    ALL=(ALL:ALL) ALL
55 user1    ALL=(ALL)    NOPASSWD: ALL  추가
56 # See sudoers(5) for more information on
```

1.1 실습 환경 구성

ubuntu에 s/w 설치

```
root@master:~# vi /etc/hosts
```

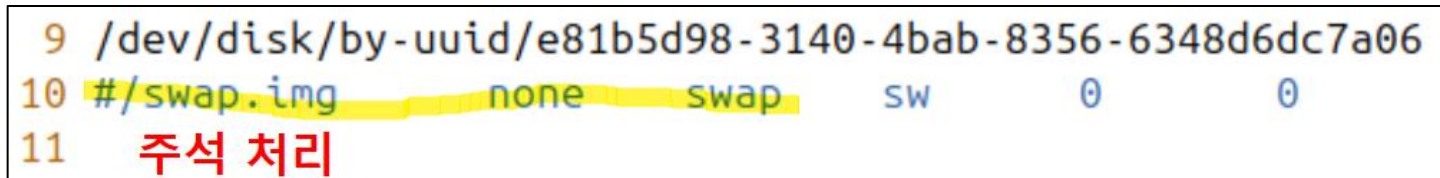


```
127.0.0.1 localhost
172.16.8.10 master
172.16.8.11 node1
172.16.8.12 node2
```

추가

```
root@master:~# swapoff -a
```

```
root@master:~# vi /etc/fstab
```



```
9 /dev/disk/by-uuid/e81b5d98-3140-4bab-8356-6348d6dc7a06
10 #/swap.img none swap sw 0 0
11 주석 처리
```

1.1 실습 환경 구성

ubuntu에 s/w 설치

```
root@master:~# systemctl disable ufw
```

```
root@master:~# systemctl stop ufw
```

```
root@master:~# systemctl start ssh
```

```
root@master:~# systemctl enable ssh
```

```
root@master:~# vi /etc/ssh/sshd_config
```

```
41 #LoginGraceTime 2m  
42 PermitRootLogin yes 수정  
43 #StrictModes yes
```

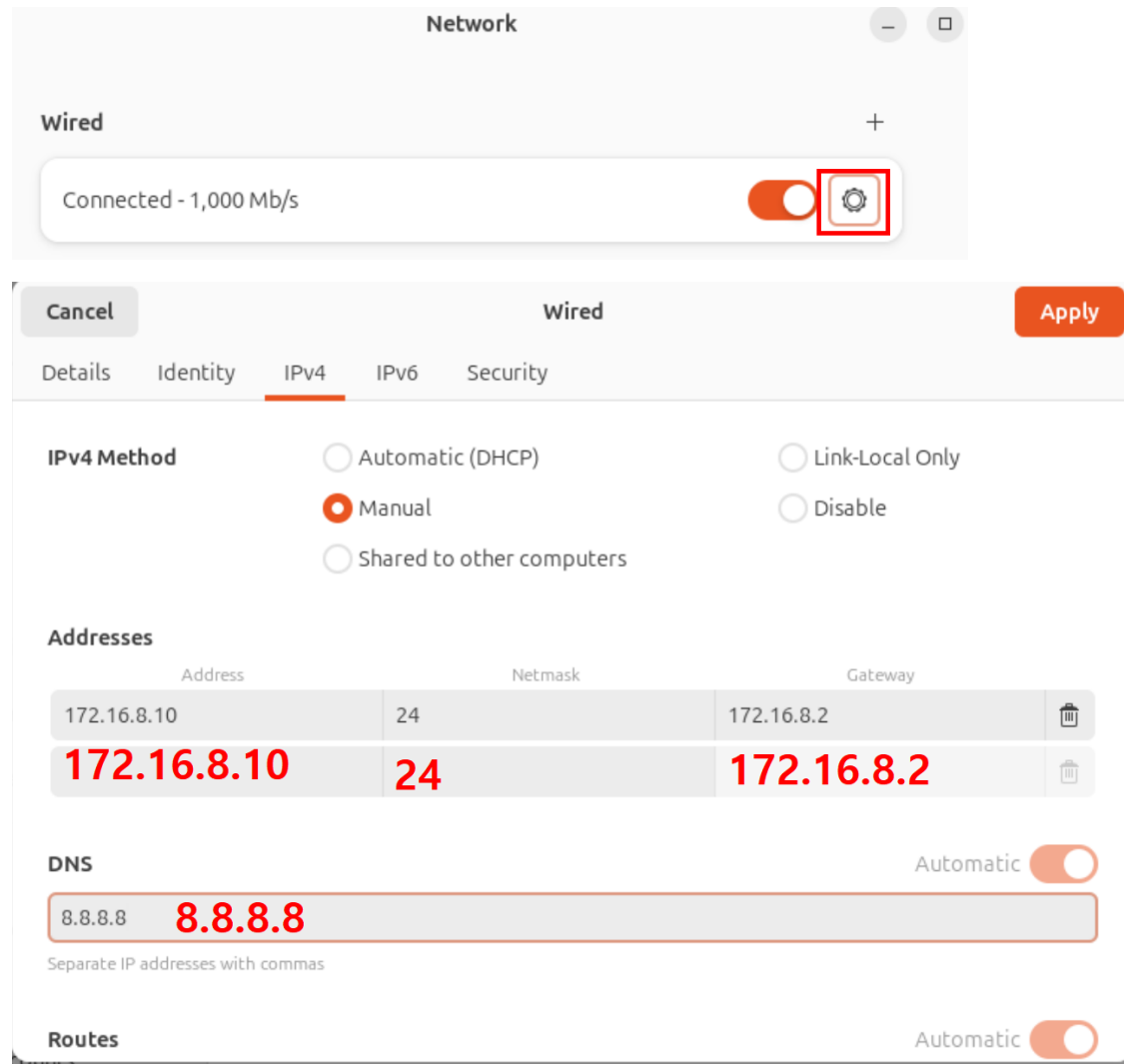
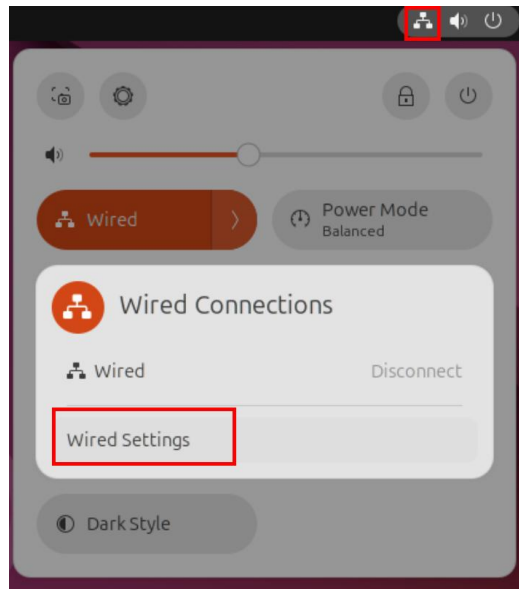
```
root@master:~# systemctl restart ssh
```

```
root@master:~# passwd root
```

패스워드 설정 (비번: 1234)

1.1 실습 환경 구성

master 노드 IP 주소 설정



1.1 실습 환경 구성

master 노드 IP 주소 설정 확인

```
root@master:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
up default qlen 1000
    link/ether 00:0c:29:2e:0f:e1 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 172.16.8.10/24 brd 172.16.8.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
```

1.1 실습 환경 구성

MobaXterm Linux 원격접속 App 설치

<https://mobaxterm.mobatek.net/download.html>

The screenshot displays the MobaXterm website's download page. On the left, there are two main sections: 'Home Edition' (Free) and 'Professional Edition' (\$69 / 49€ per user*). The 'Home Edition' section lists features like Full X server and SSH support, Remote desktop (RDP, VNC, Xdmcp), Remote terminal (SSH, telnet, rlogin, Mosh), X11-Forwarding, Automatic SFTP browser, Master password protection, Plugins support, Portable and installer versions, Full documentation, Max. 12 sessions, Max. 2 SSH tunnels, Max. 4 macros, and Max. 360 seconds for Tftp, Nfs and Cron. A 'Download now' button is highlighted with a red box. The 'Professional Edition' section lists additional features like Customizable startup message and logo, Modify your profile script, Remove unwanted games, screensaver or tools, Unlimited number of sessions, Unlimited number of tunnels and macros, Unlimited run time for network daemons, Enhanced security settings, 12-months updates included, Deployment inside company, and Lifetime right to use. A 'Subscribe online / Get a quote' button is also present. On the right, the 'MobaXterm Home Edition' section shows the current version (v26.3) with two download options: 'Portable edition' (blue button) and 'Installer edition' (green button). Both buttons are highlighted with red boxes. Below this, a list of files is shown: MobaXterm.ini, MobaXterm backup.zip, CygUtils.plugin, CygUtils64.plugin, and MobaXterm_Personal_26.3.exe. The last file is highlighted with a red box.

Home Edition

Free

- Full **X server** and **SSH** support
- Remote desktop (RDP, VNC, Xdmcp)
- Remote terminal (SSH, telnet, rlogin, Mosh)
- X11-Forwarding
- Automatic SFTP browser
- Master password protection
- Plugins support
- Portable and installer versions
- Full documentation
- Max. **12** sessions
- Max. **2** SSH tunnels
- Max. **4** macros
- Max. **360** seconds for Tftp, Nfs and Cron

Download now

Professional Edition

\$69 / 49€ per user*

* Excluding tax. Volume discounts [available](#)

Every feature from Home Edition +

- Customize your startup message and logo
- Modify your profile script
- Remove unwanted games, screensaver or tools
- Unlimited number of sessions
- Unlimited number of tunnels and macros
- Unlimited run time for network daemons
- Enhanced security settings
- 12-months updates included
- Deployment inside company
- Lifetime right to use

Subscribe online / Get a quote

MobaXterm Home Edition

Download MobaXterm Home Edition (current version):

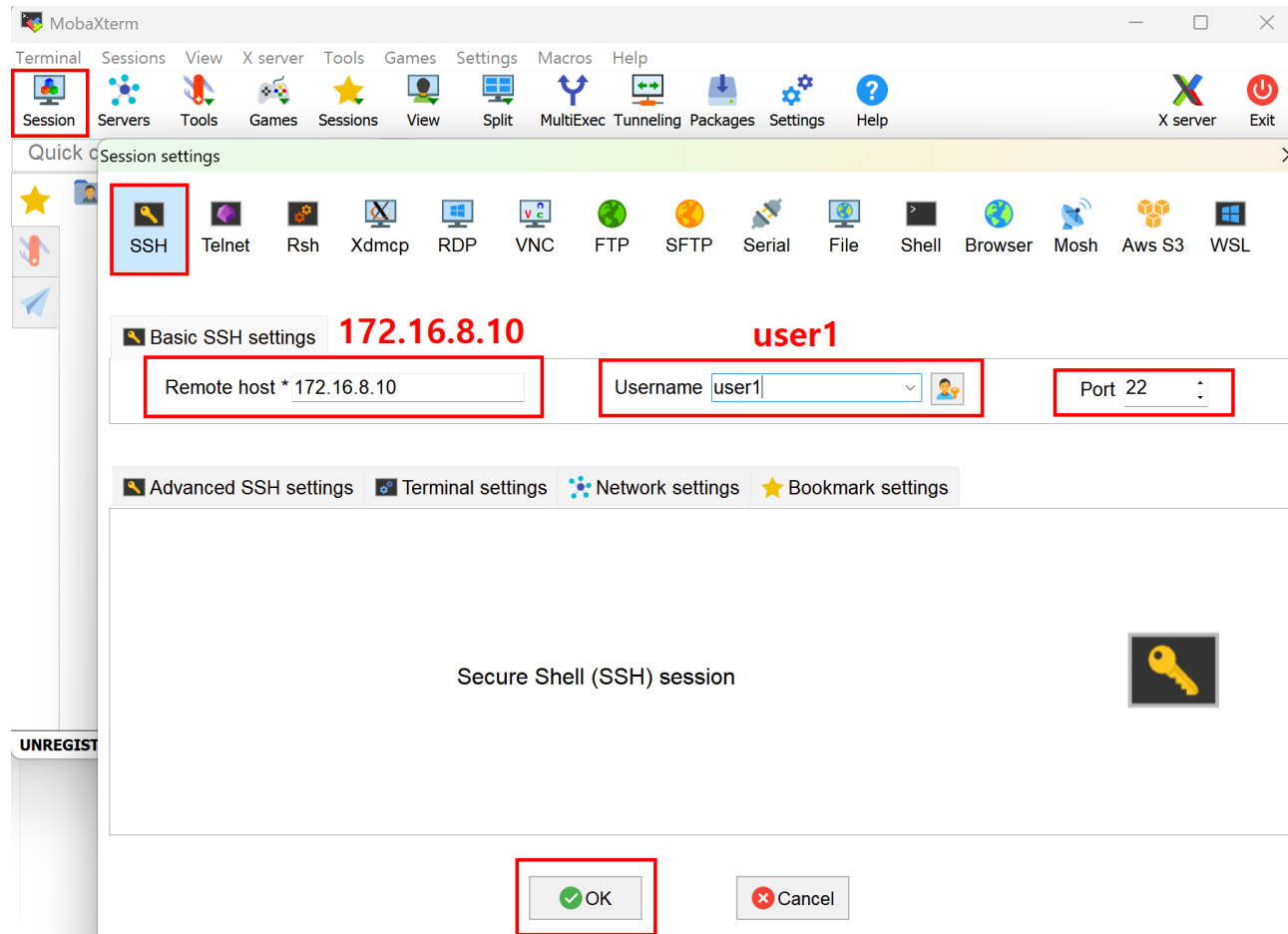
MobaXterm Home Edition v26.3 (Portable edition)

MobaXterm Home Edition v26.3 (Installer edition)

- MobaXterm.ini
- MobaXterm backup.zip
- CygUtils.plugin
- CygUtils64.plugin
- MobaXterm_Personal_26.3.exe**

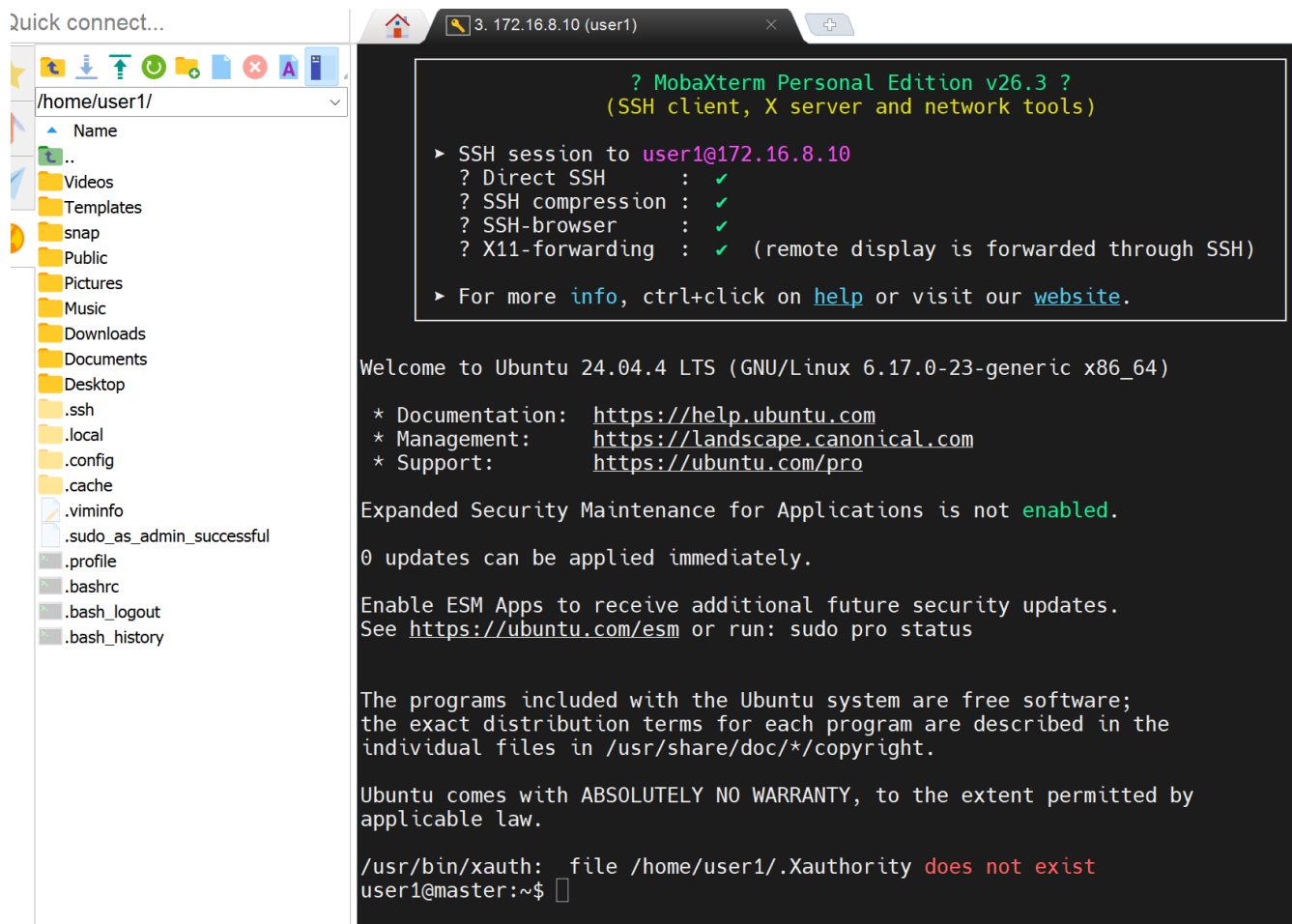
1.1 실습 환경 구성

MobaXterm 으로 원격접속 하기



1.1 실습 환경 구성

MobaXterm 으로 원격접속 하기



1.1 실습 환경 구성

pem 파일 이용한 Linux에 로그인하기

공개키(public key) 와 개인키 (private key) 생성하기

```
ssh-keygen -q -N "" -f master_key.pem
```

공개키의 내용을 authorized_keys 파일에 기록한다

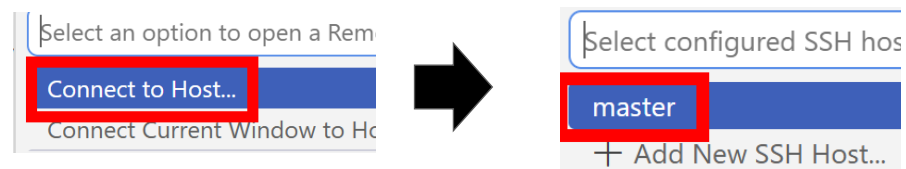
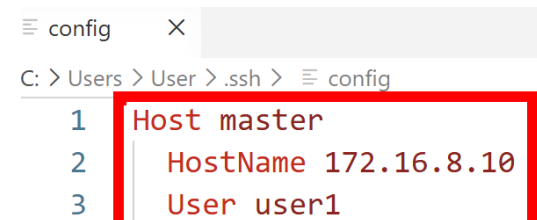
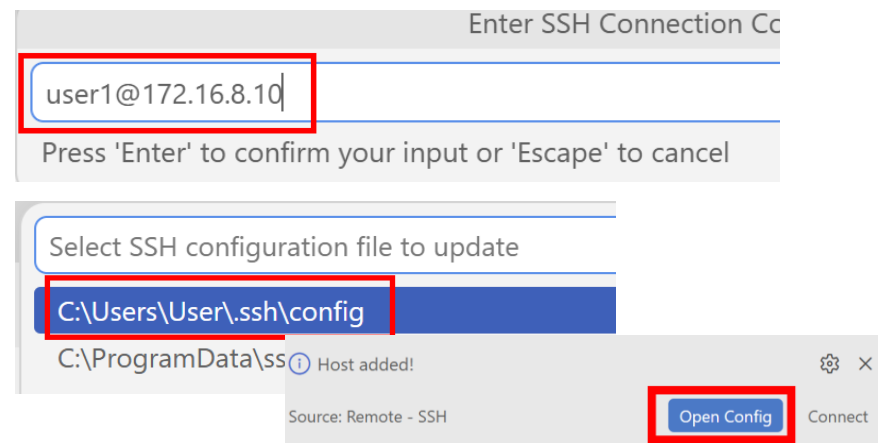
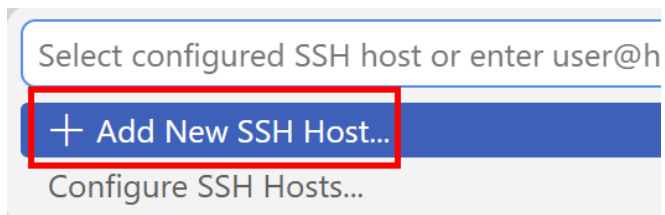
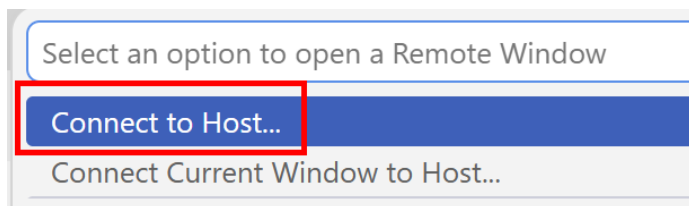
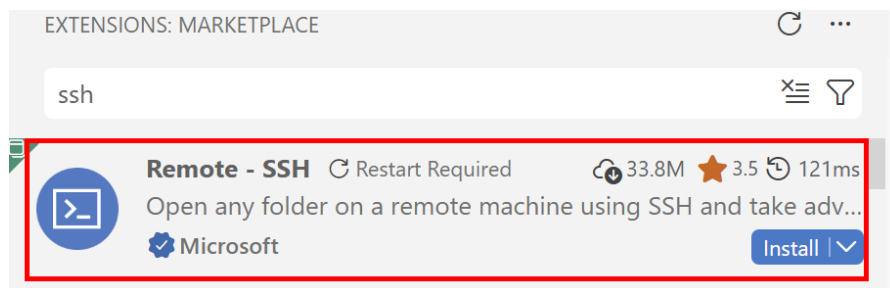
```
cat master_key.pem.pub >> ~/.ssh/authorized_keys
```

권한 낮추기

```
chmod 600 ~/.ssh/authorized_keys
```

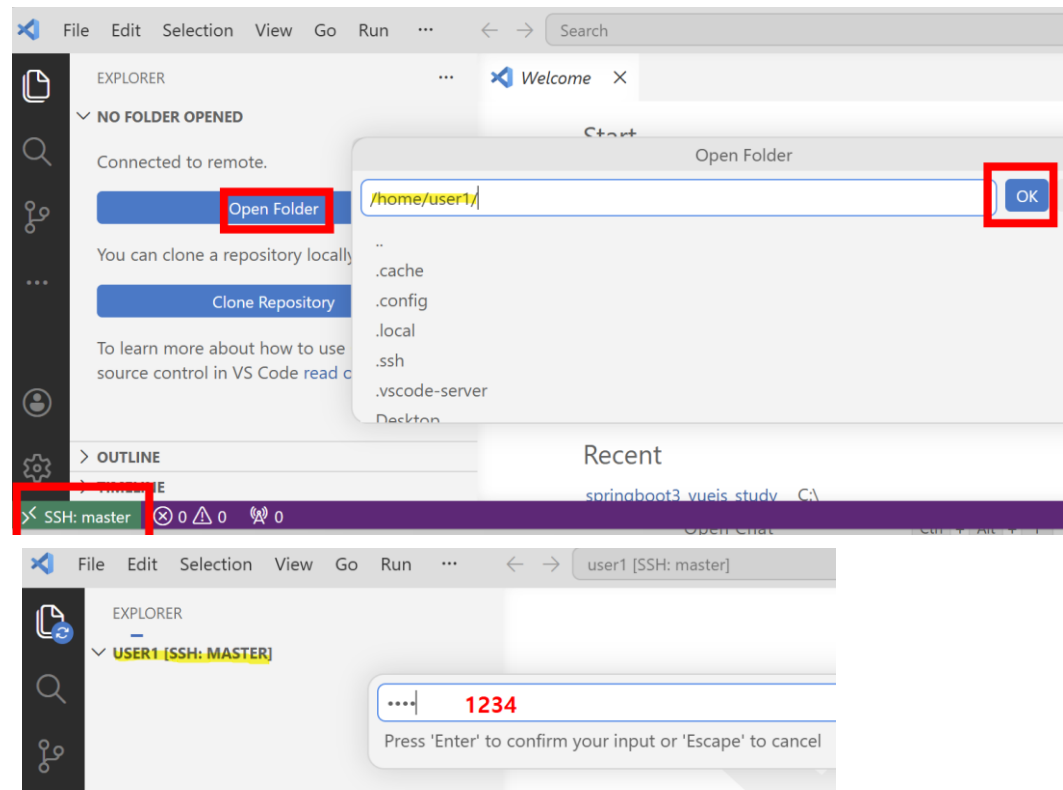
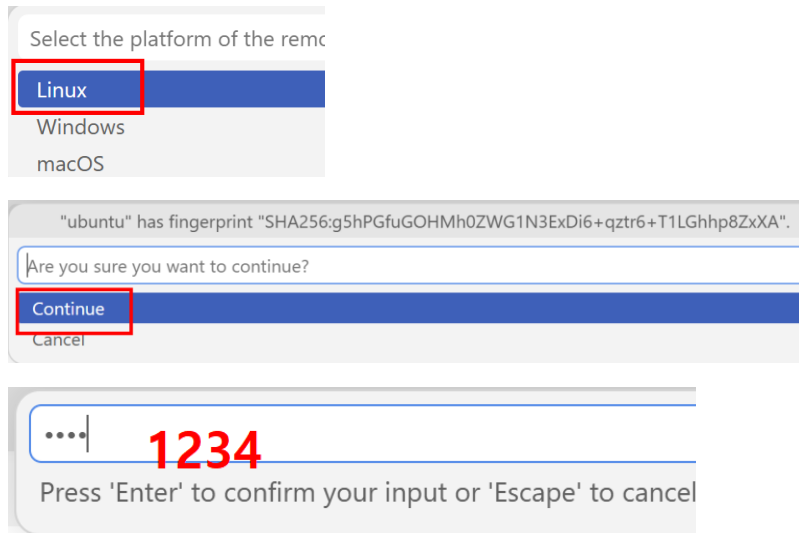
1.1 실습 환경 구성

VSC에서 Linux에 로그인하기



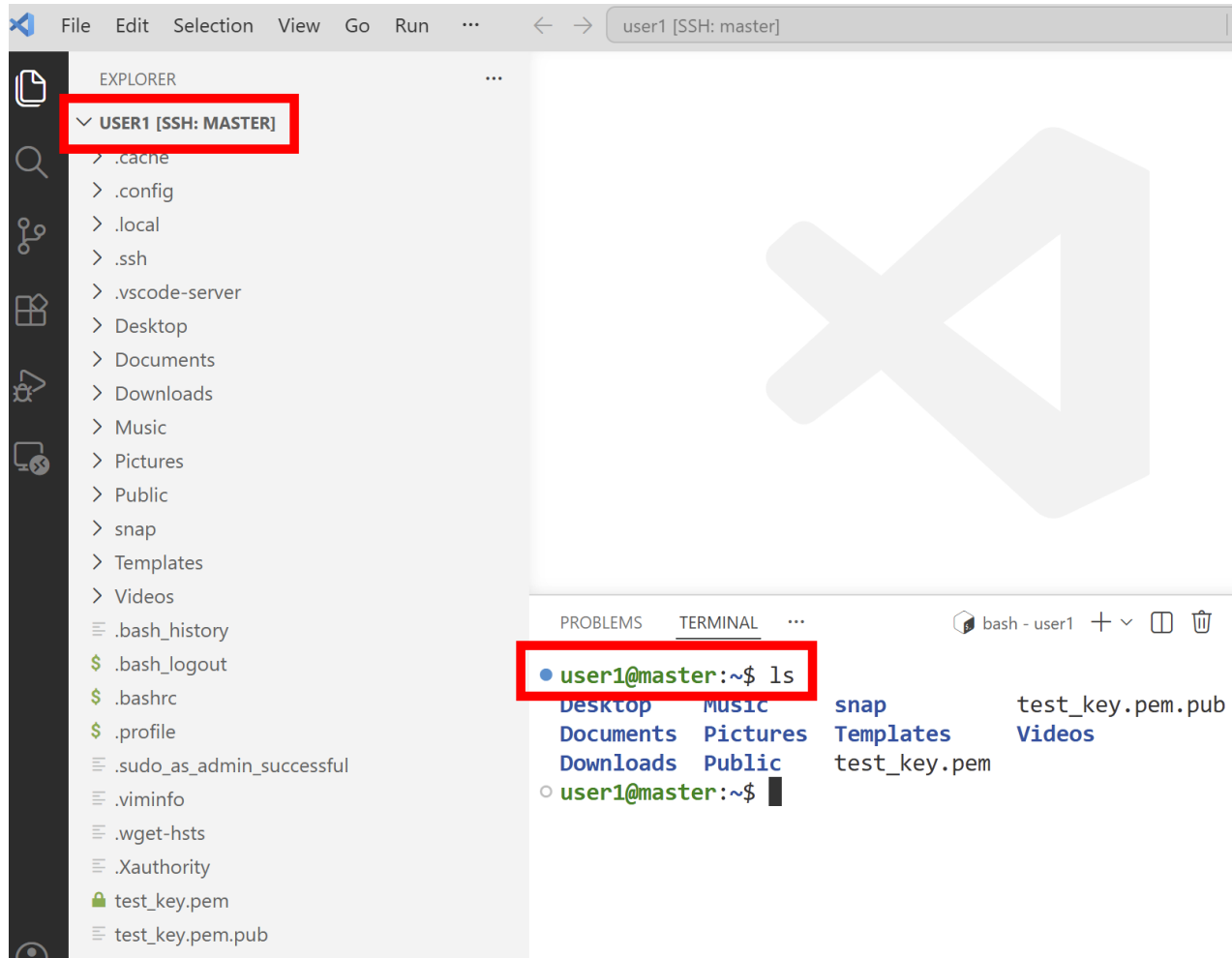
1.1 실습 환경 구성

VSC에서 Linux에 로그인하기



1.1 실습 환경 구성

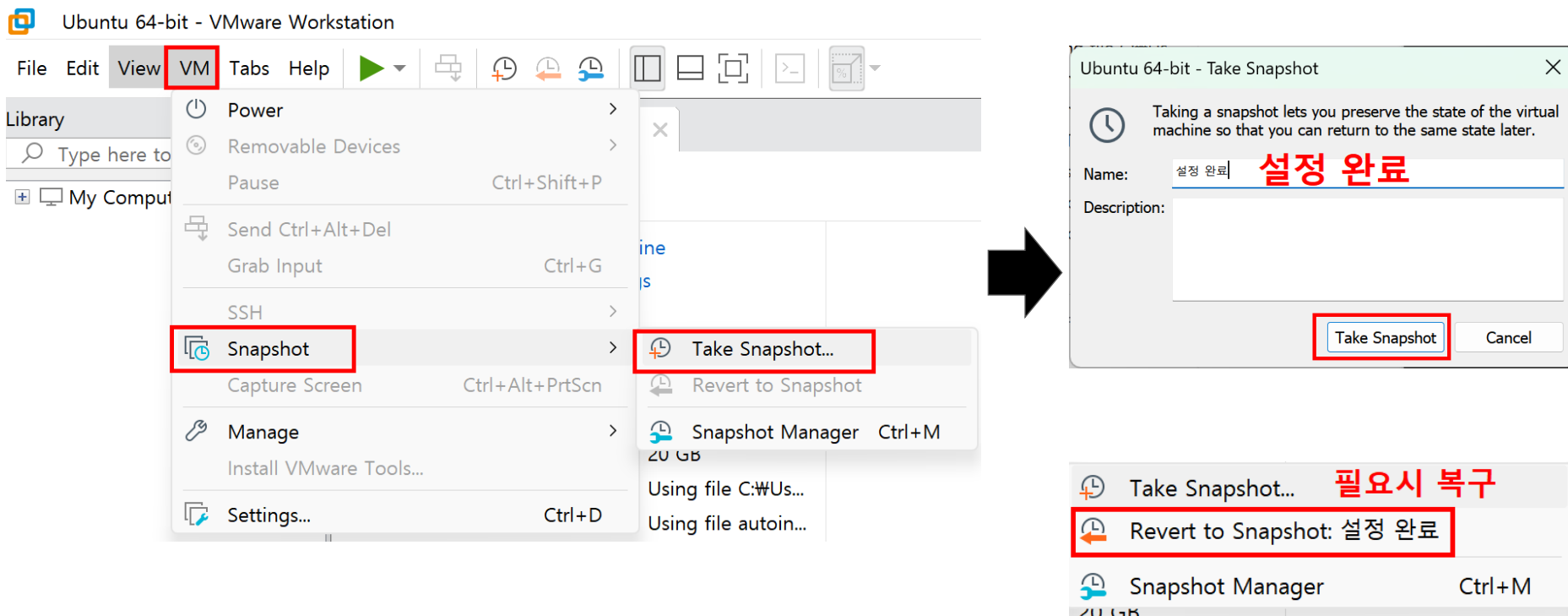
VSC에서 Linux에 로그인하기



1.1 실습 환경 구성

스냅샷 만들기

1. 먼저 ubuntu 종료
2. VMWare > VM 메뉴 > Snapshot > Take Snapshot... 선택



1.2 Docker 설치

Docker 설치

Add Docker's official GPG key:

```
sudo apt-get update -y
```

```
sudo apt-get install ca-certificates curl -y
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Add the repository to Apt sources:

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
```

```
https://download.docker.com/linux/ubuntu \
```

```
${. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}" stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update -y
```

1.2 Docker 설치

Docker 설치

설치 가능한 Docker 버전 확인 (선택 사항)

```
apt-cache madison docker-ce
```

Docker 28.5.1 정확히 지정 설치

```
sudo apt-get install -y \  
docker-ce=5:28.5.1-1~ubuntu.24.04~noble \  
docker-ce-cli=5:28.5.1-1~ubuntu.24.04~noble \  
containerd.io \  
docker-buildx-plugin \  
docker-compose-plugin
```

Docker 자동 업그레이드 방지 (버전 고정)

```
sudo apt-mark hold docker-ce docker-ce-cli
```

1.3 K8s 설치

K8s 설치

```
# kubernetes -cri (containerd, cri-o | cri-dockerd) - dockerd
```

cri-dockerd 는 cri 환경에서 기존의 dockerd 를 사용할수 있도록 해준다.

```
wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.15/cri-dockerd-0.3.15.amd64.tgz
```

```
tar xvf cri-dockerd-0.3.15.amd64.tgz
```

```
sudo mv cri-dockerd/cri-dockerd /usr/local/bin/
```

[확인]

```
su -
```

Password: 1234

```
root@master:~# which cri-dockerd
```

```
    /usr/local/bin/cri-dockerd
```

```
root@master:~#
```

1.3 K8s 설치

K8s 설치

[서비스 등록 및 실행]

```
wget https://raw.githubusercontent.com/Mirantis/cni-dockerd/master/packaging/systemd/cni-docker.service
```

```
wget https://raw.githubusercontent.com/Mirantis/cni-dockerd/master/packaging/systemd/cni-docker.socket
```

```
sudo mv cni-docker.socket cni-docker.service /etc/systemd/system/
```

```
sudo sed -i -e 's,/usr/bin/cni-dockerd,/usr/local/bin/cni-dockerd,' /etc/systemd/system/cni-docker.service
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable cni-docker.service --now
```

```
sudo systemctl enable cni-docker.socket --now
```

[확인]

```
root@master:~# systemctl status cni-docker
```

- cni-docker.service - CRI Interface for Docker Application Container Engine

Loaded: loaded (/etc/systemd/system/cni-docker.service; enabled; preset: enabled)

Active: active (running) since Fri 2026-05-01 22:48:46 KST; 1min 31s ago

1.3 K8s 설치

리소스 관리 통합

```
cat <<EOF | sudo tee /etc/docker/daemon.json
```

```
{  
  "exec-opts": ["native.cgroupdriver=systemd"],  
  "log-driver": "json-file",  
  "log-opts": {  
    "max-size": "100m"  
  },  
  "storage-driver": "overlay2"  
}
```

EOF

```
sudo systemctl restart docker && sudo systemctl restart cri-docker
```

[확인]

```
root@master:~# sudo docker info | grep Cgroup
```

Cgroup Driver: systemd

Cgroup Version: 2

1.3 K8s 설치

K8s 동작을 위한 패키지 설치

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
```

```
sudo mkdir -p -m 755 /etc/apt/keyrings
```

```
sudo curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o  
/etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
sudo echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/  
' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

다음 작업들은 시간이 약간 걸림.

```
sudo apt-get update
```

```
sudo apt-get install -y kubelet kubeadm kubectl
```

```
sudo apt-mark hold kubelet kubeadm kubectl # upgrade 를 통해 버전업 되지 않도록
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart kubelet
```

1.4 node 복제

생성된 인스턴스를 복제하여 3대의 노드로 작성.

1. 인스턴스 종료(Power off).
2. K8s_master 탭 > 오른쪽 클릭 > Manage > Clone > 다음 >
The current state in the virtual machine > Create a full clone (중요) >
virtual machine name: node1 입력하고 마침 클릭.
3. VMWare 목록에 node1이 보임.
4. node1 리소스 조정하기. (Memory: 2GB, Processor: 2)
5. 동일한 방법으로 node2와 node3도 만들자.

1.4 node 복제

6. clone이 모두 끝났으면 각 노드별로 IP, hostname을 수정해야 됨.

ex. node1 변경 예

1) node1 실행

2) 이름변경: 환경설정 > System 항목 > About > Device Name 항목을 node1 변경하고 옆 체크 클릭(****)

3) IP변경: 환경설정 > Network > 설정 > IPv4 > Manual >

Address: 172.16.8.11

netmask: 24

Gateway: 172.16.8.2

DNS: 8.8.8.8

7. 리소스 줄이기 위해서 init 3으로 부팅하기

- ssh로 접속하여 다음 명령어 실행

sudo su -

systemctl set-default multi-user.target && reboot

1.5 Docker 로그인

패스워드가 아니라 토큰으로 로그인 하기.

- 1) 계정으로 로그인
- 2) Account Settings 선택
- 3) 메뉴 하단에서 Personal access tokens
- 4) Generate new token 선택

Access token description: K8s 실습용 토큰

Expiration data: 90days

Optional: Read,Write, Deelte

Generate 버튼 클릭

1.5 Docker 로그인

패스워드가 아니라 토큰으로 로그인 하기.

5) Copy access token

Run: `docker login -u kyongyeolin`

token: `dckr_pat_o4umnXm3Zn2sf1GTZdk-tDGd5AM`

6) `$ docker login -u kyongyeolin(계정명)`

password: token값

7) 위 토큰을 이용하여 모든 노드(master,node1, node2, node3)에서 로그인한다.

1.6 K8s 설정

master노드에서 해야 될 작업.

```
sudo su -
```

```
modprobe br_netfilter
```

```
echo 1 > /proc/sys/net/bridge/bridge-nf-call-iptables
```

```
kubeadm config images pull --cri-socket unix:///run/cri-dockerd.sock
```

```
kubeadm init --cri-socket /var/run/cri-dockerd.sock
```

```
mkdir -p $HOME/.kube
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

```
...
```

```
kubeadm join 172.16.8.10:6443 --token 4ra5uh.dr48outab7hgo9jr \
```

```
--discovery-token-ca-cert-hash sha256:6fccea1267cf541ff2e7596552a9e9e6b13c5ac860757e306fa16d2c8a2ac9
```

1.6 K8s 설정

master노드에서 해야 될 작업.

pod 의 ip 대역(172.18.x.x)도 같이 설정하고 싶으면 아래와 같이 옵션을 추가한다 (실습에서는 추가안함)

나중에 node간 연결시 Calico 사용할때는 자동으로 대역을 설정해줌. 따라서 여기서는 실습안함

kubeadm init --cri-socket /var/run/cri-dockerd.sock --pod-network-cidr=172.18.0.0/16

1.6 K8s 설정

node1, node2 에서 해야 될 작업.

1. 브릿지 네트워크 필터 활성화 (Pod 간 통신을 위해 필수)

```
sudo su -
```

```
modprobe br_netfilter
```

2. iptables가 브릿지된 트래픽을 볼 수 있도록 커널 파라미터 수정

```
echo 1 > /proc/sys/net/bridge/bridge-nf-call-iptables
```

3. 모든 worker 노드를 마스터 노드에 연결

주의할점은 --cri-socket unix:///var/run/cri-dockerd.sock 옵션은 반드시 추가한다.(*****)

위 master 실행결과 명령어 값 + 옵션추가해서 해야됨 (*****)

```
kubeadm join 172.16.8.10:6443 --token 4ra5uh.dr48outab7hgo9jr \
```

```
--discovery-token-ca-cert-hash sha256:6fccea1267cfeb541ff2e7596552a9e9e6b13c5ac860757e306fa16d2c8a2ac9\
```

```
--cri-socket unix:///var/run/cri-dockerd.sock
```

1.6 K8s 설정

node1, node2 에서 해야 될 작업.

4. master에서 연결정보 확인

kubectl get node # 모두 NotReady 상태임.

```
root@master:~# kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	NotReady	control-plane	48m	v1.29.15
node1	NotReady	<none>	59s	v1.29.15
node2	NotReady	<none>	30s	v1.29.15

현재는 CNI가 설치되어 있지 않아 NotReady 상태임.

1.7 Calico CNI 이용한 노드 연결

Calico CNI 연결.

1. 모든 노드(master, node1, node2)에서 다음 이미지를 pull 받음

```
docker pull docker.io/calico/cni:v3.27.1
```

```
docker pull docker.io/calico/node:v3.27.1
```

```
docker pull docker.io/calico/kube-controllers:v3.27.1
```

2. master 에서 Calico를 설치하고 확인한다.

```
wget https://raw.githubusercontent.com/projectcalico/calico/v3.27.1/manifests/calico.yaml
```

```
kubectl apply -f calico.yaml
```

```
kubectl get node # 모두 ready 상태로 변경됨.
```

1.8 실습 편의를 위한 추가 작업

kubectl 별칭 설정

1) 먼저 vi 편집기를 열어서

```
vi ~/.bashrc
```

2) 아래 2줄을 추가하고 저장후 빠져나온다.

```
export KUBECONFIG=/etc/kubernetes/admin.conf # root > kubernetes admin  
alias k='kubectl'
```

3) 바로 적용되도록 다시 로딩 시킨다

```
source ~/.bashrc
```

4) 별칭으로 명령어 실행

```
root@master:~# k get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	108m	v1.29.15

```
..
```

수고하셨습니다.